

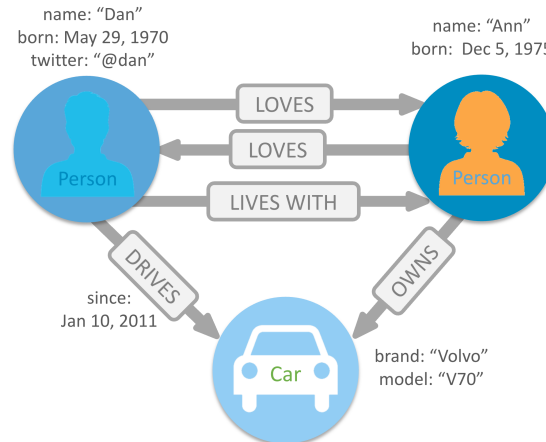
COMPSCI 752

Big Data Management

Instructor: Philipp Skavantzios (philipp.skavantzios@auckland.ac.nz)

expressive.

Graph Queries



The rich world of Graph Query Languages

**What are the titles of
movies that Tom
Hanks acted in?**

The rich world of Graph Query Languages

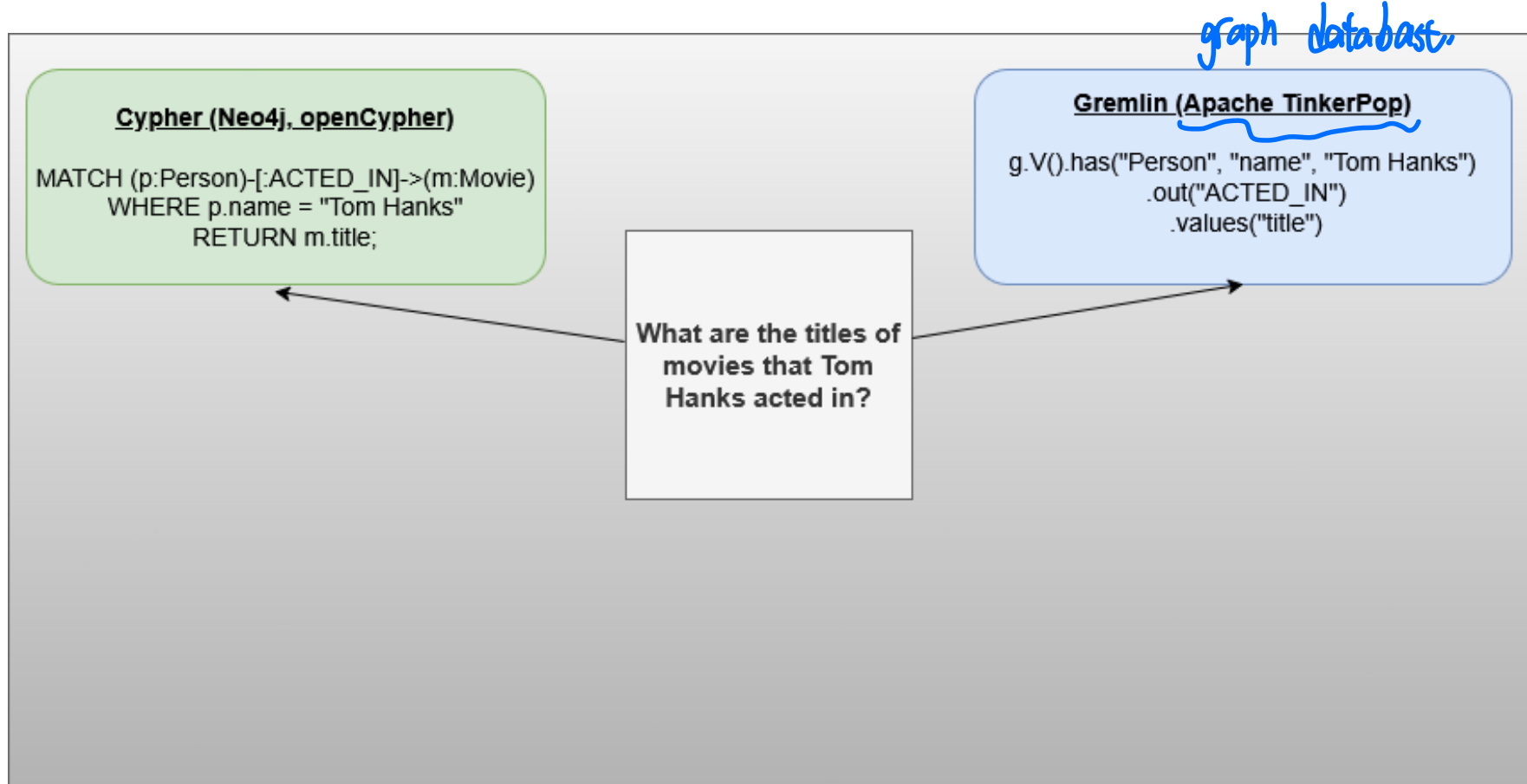
Cypher (Neo4j, openCypher)

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
WHERE p.name = "Tom Hanks"
RETURN m.title;
```

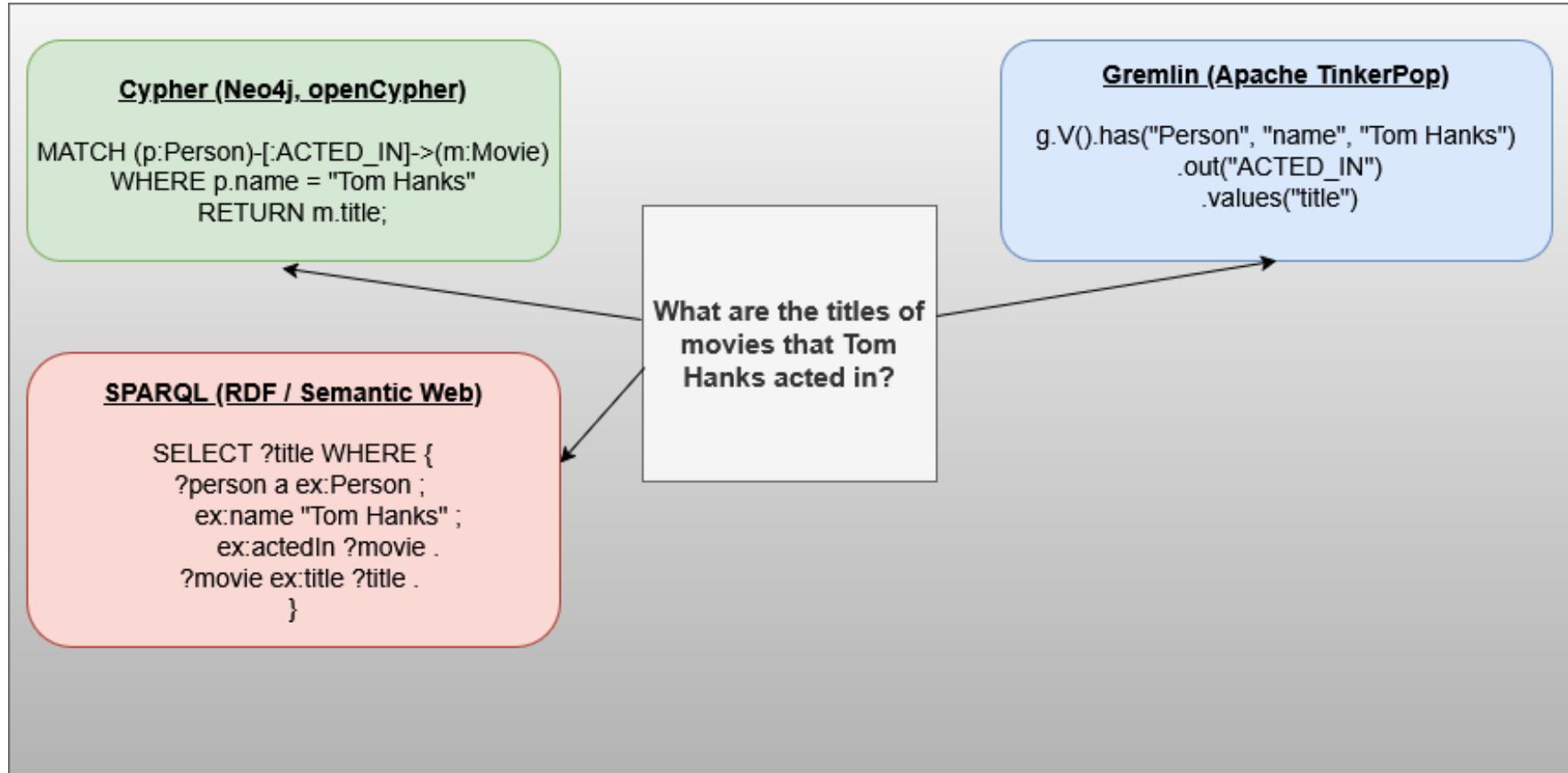
What are the titles of
movies that Tom
Hanks acted in?

A black arrow points from the text box on the right to the Cypher query box on the left, indicating that the query is the answer to the question.

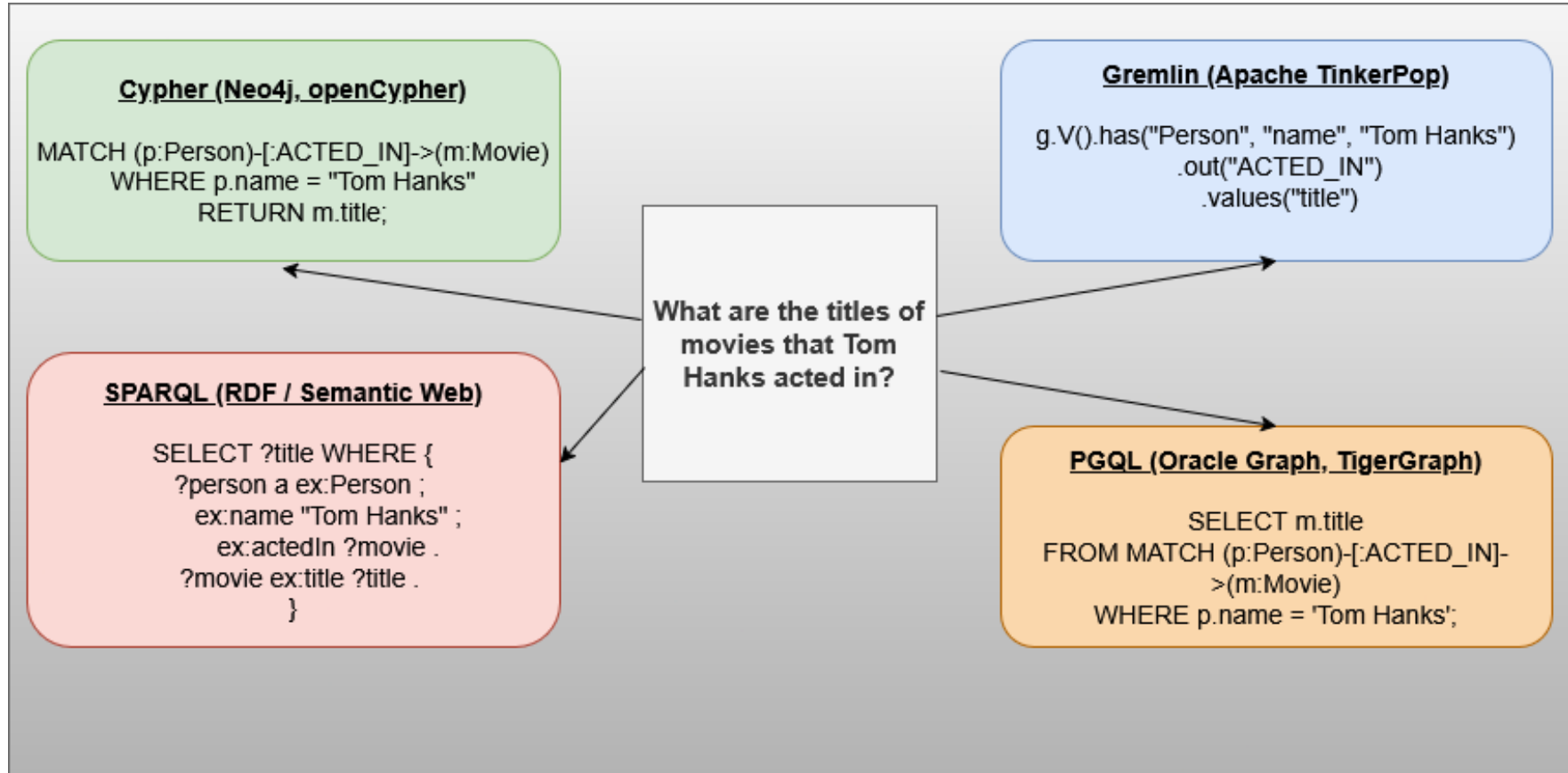
The rich world of Graph Query Languages



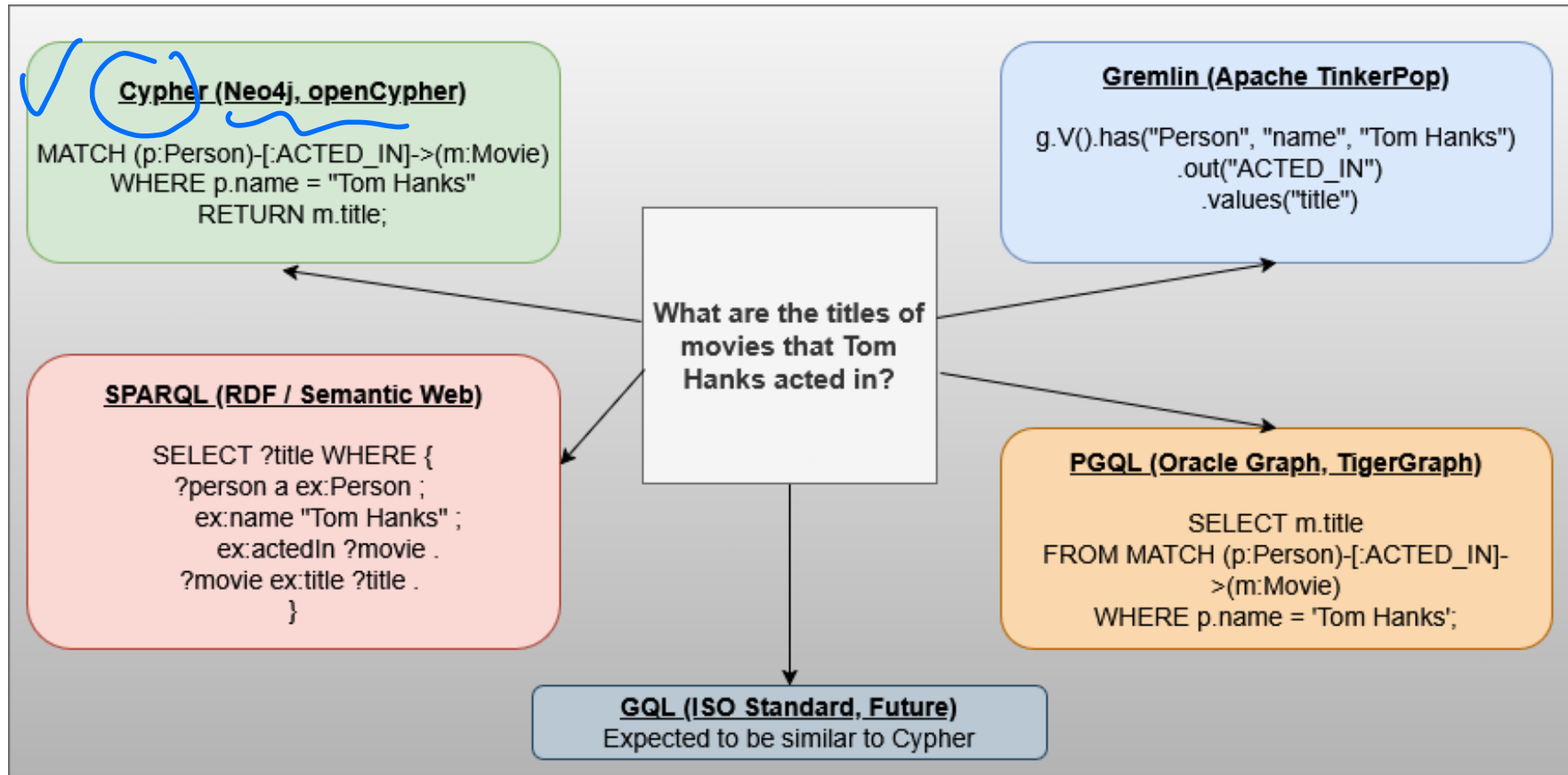
The rich world of Graph Query Languages



The rich world of Graph Query Languages



The rich world of Graph Query Languages



So many Graph Query Languages. How did we get here?

So many Graph Query Languages. How did we get here?

Let us recall the Relational Data Model.

Foundations of SQL

Relational algebra

$\pi_{\text{title, year}}(\sigma_{\text{last_name}='Kurosawa'}(\sigma_{\text{first_name}='Akira'}(\text{MOVIE} \bowtie \text{DIRECTOR} \bowtie \text{PERSON})))$

Foundations of SQL

Relational algebra

$$\pi_{\text{title, year}}(\sigma_{\text{last_name}='Kurosawa'}(\sigma_{\text{first_name}='Akira'}(\text{MOVIE} \bowtie \text{DIRECTOR} \bowtie \text{PERSON})))$$

微积分

Relational calculus

$$\{(x_{\text{title}}, x_{\text{year}}) \mid \exists x_{\text{country}}, x_{\text{run_time}}, x_{\text{genre}}, x_{\text{id}}, x_{\text{year_born}} (\text{MOVIE}(x_{\text{title}}, x_{\text{year}}, x_{\text{country}}, x_{\text{run_time}}, x_{\text{genre}}) \wedge \text{DIRECTOR}(x_{\text{id}}, x_{\text{title}}, x_{\text{year}}) \wedge \text{PERSON}(x_{\text{id}}, 'Akira', 'Kurosawa', x_{\text{year_born}}))\}$$

Foundations of SQL

Relational algebra

$$\pi_{\text{title, year}}(\sigma_{\text{last_name}='Kurosawa'}(\sigma_{\text{first_name}='Akira'}(\text{MOVIE} \bowtie \text{DIRECTOR} \bowtie \text{PERSON})))$$

SQL

```
SELECT  m.title, m.year
FROM    MOVIE m, DIRECTOR d, PERSON p
WHERE   m.title=d.title AND m.year=d.year AND
        d.id=p.id AND p.first_name='Akira' AND p.last_name='Kurosawa';
```

Relational calculus

$$\{(x_{\text{title}}, x_{\text{year}}) \mid \exists x_{\text{country}}, x_{\text{run_time}}, x_{\text{genre}}, x_{\text{id}}, x_{\text{year_born}} (\text{MOVIE}(x_{\text{title}}, x_{\text{year}}, x_{\text{country}}, x_{\text{run_time}}, x_{\text{genre}}) \wedge \text{DIRECTOR}(x_{\text{id}}, x_{\text{title}}, x_{\text{year}}) \wedge \text{PERSON}(x_{\text{id}}, \text{'Akira'}, \text{'Kurosawa'}, x_{\text{year_born}}))\}$$

Now what does this look like for Graph Query Languages?

Query languages

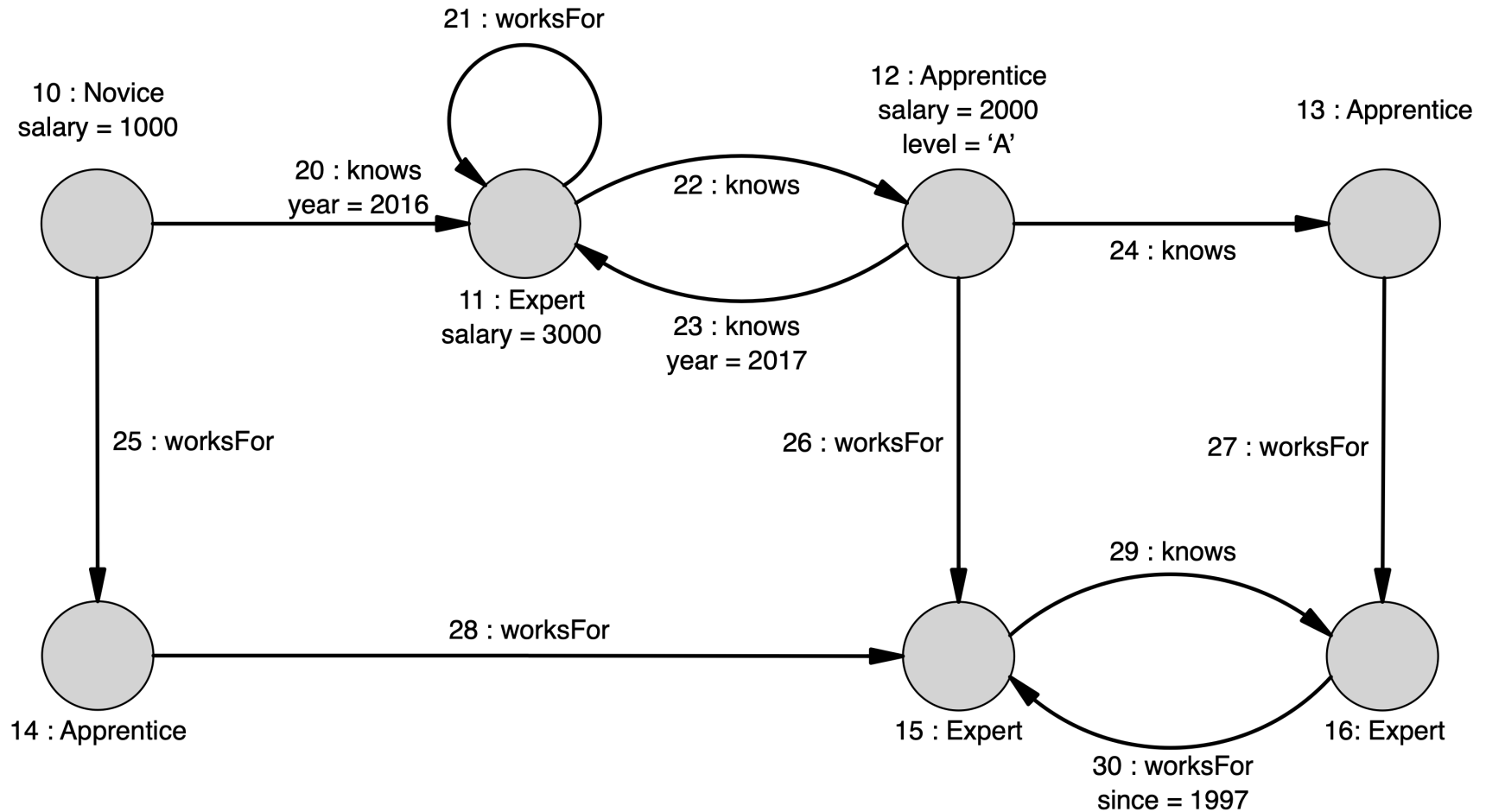
Language to provide uniform access to data across domains

Language to provide uniform access to data across domains

Aims

- Core language functionalities of graph navigation queries and (unions of) conjunctions of navigational queries
- Algebra for more efficient query evaluation
- Calculus as means to express sound queries declaratively: use computationally well-behaved yet expressive fragment of Datalog for querying graphs

Query Languages



Regular Path Queries

- Example: find direct and indirect connections in social and biological networks
- *path navigation* is a core ingredient of graph querying

Regular Path Queries

- Example: find direct and indirect connections in social and biological networks
- path navigation is a core ingredient of graph querying

Regular Path Queries (RPQ) 可达性.

- Enable us to express such reachability queries
- They retrieve all pairs of vertices that
 - are connected by at least one path where
 - the sequence of edge labels along the path forms a word in
 - the language of a given regular expression over the graph's edge labels

Syntax of Regular Path Queries

Definition

The *regular path queries* are all and only those expressions recursively generated as follows:

- If $a \in \mathcal{L}$, then $a \in RPQ$ ($\mathcal{L} \subseteq RPQ$),
- If $e \in RPQ$, then $(e)^- \in RPQ$,
- If $e \in RPQ$, then $(e)^+ \in RPQ$,
- If $e, f \in RPQ$, then $(e)/(f) \in RPQ$,
- If $e, f \in RPQ$, then $(e) + (f) \in RPQ$.

Syntax of Regular Path Queries

Definition

The *regular path queries* are all and only those expressions recursively generated as follows:

- If $a \in \mathcal{L}$, then $a \in RPQ$ ($\mathcal{L} \subseteq RPQ$),
- If $e \in RPQ$, then $(e)^- \in RPQ$,
- If $e \in RPQ$, then $(e)^+ \in RPQ$,
- If $e, f \in RPQ$, then $(e)/(f) \in RPQ$,
- If $e, f \in RPQ$, then $(e) + (f) \in RPQ$.

省略括号

We omit parentheses if this does not cause ambiguities

Example for the Syntax of a Regular Path Query

$$\mathcal{L} = \{ :knows, :worksFor \}$$

Example for the Syntax of a Regular Path Query

$$\mathcal{L} = \{ :knows, :worksFor \}$$

Example (Regular Path Queries over $\mathcal{L}$)

- $:knows \in RPQ$
- $:worksFor \in RPQ$

Example for the Syntax of a Regular Path Query

$$\mathcal{L} = \{ :knows, :worksFor \}$$

Example (Regular Path Queries over $\mathcal{L}$)

- $:knows \in RPQ$
 - $:worksFor \in RPQ$
 - $:knows^+ \in RPQ$
- 连串的*

Example for the Syntax of a Regular Path Query

$$\mathcal{L} = \{ :knows, :worksFor \}$$

Example (Regular Path Queries over $\mathcal{L}$)

- $:knows \in RPQ$
- $:worksFor \in RPQ$
- $:knows^+ \in RPQ$
- $:worksFor/:knows^+ \in RPQ$

Example for the Syntax of a Regular Path Query

$$\mathcal{L} = \{ :knows, :worksFor \}$$

Example (Regular Path Queries over $\mathcal{L}$)

- $:knows \in RPQ$
- $:worksFor \in RPQ$
- $:knows^+ \in RPQ$
- $:worksFor/:knows^+ \in RPQ$
- $:knows/:worksFor/:knows^+ \in RPQ$

Semantics of Regular Path Queries - Informal Explanation

RPQ as a Query Algebra

The *regular path queries* enable us to:

- Select all pairs of vertices that share an edge label ($\mathcal{L} \subseteq RPQ$),
- Take the inverse of a set of paths $(e)^- \in RPQ$,
- Take the transitive closure of a set of paths $(e)^+ \in RPQ$,
- Concatenate paths from a set of paths $(e)/(f) \in RPQ$,
- Form the union of two sets of paths $(e) + (f) \in RPQ$

Formal Semantics of Regular Path Queries

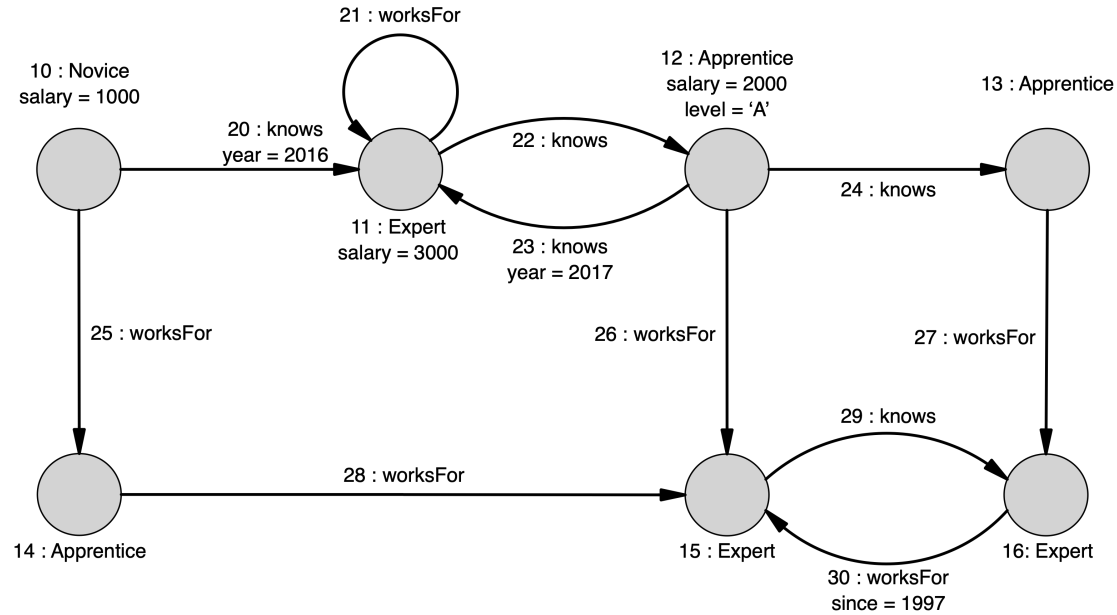
$G = (V, E, \overset{\text{direction}}{\eta}, \lambda, \nu)$ denotes a property graph

Definition

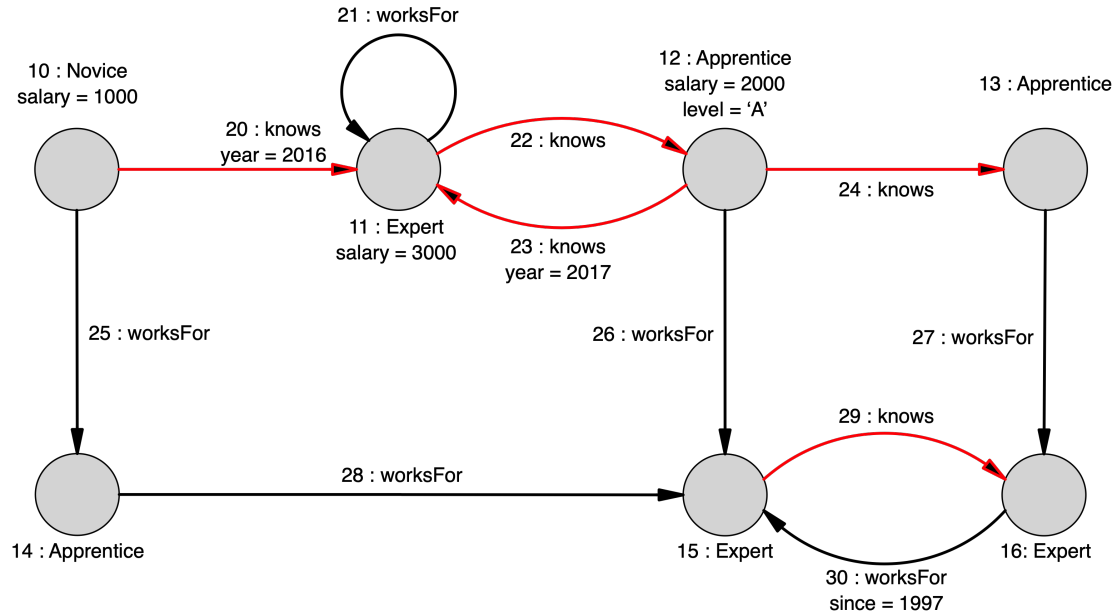
The *semantics* of evaluating an expression $g \in RPQ$ over G is the set of vertex pairs $\llbracket g \rrbracket_G \subseteq V \times V$, recursively defined as follows:

- If $g = a \in \mathcal{L}$, then $\llbracket g \rrbracket_G = \{(s, t) \mid \exists e \in E (\overset{\text{trans } v \rightarrow v \text{ pair}}{\eta(e) = (s, t)} \wedge a \in \overset{\text{direction.}}{\lambda}(e))\}$, $\overset{\text{label}}{\nu}$
- If $g = (e)^- \in RPQ$, then $\llbracket g \rrbracket_G = \{(t, s) \mid (s, t) \in \llbracket e \rrbracket_G\}$,
- If $g = (e)^+ \in RPQ$, then $\llbracket g \rrbracket_G = TC(\llbracket e \rrbracket_G)$, where $TC(r)$ denotes the transitive closure for a binary relation r ,
- If $g = \overset{\text{连接}}{(e) \text{ / } (f)} \in RPQ$, then $\llbracket g \rrbracket_G = \{(s, t) \mid \exists u \in V ((s, u) \in \llbracket e \rrbracket_G \wedge (u, t) \in \llbracket f \rrbracket_G)\}$,
- If $g = (e) + (f) \in RPQ$, then $\llbracket g \rrbracket_G = \llbracket e \rrbracket_G \cup \llbracket f \rrbracket_G$.

Example for Evaluating a Regular Path Query on a Property Graph



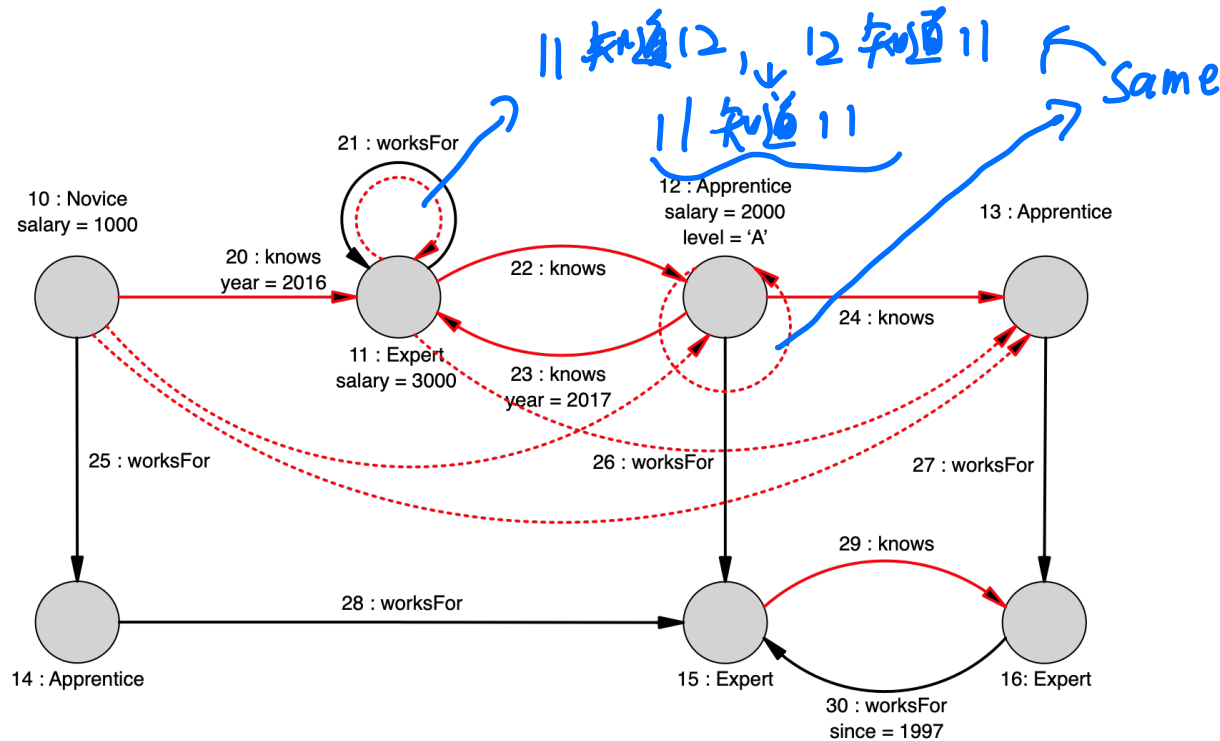
Example for Evaluating a Regular Path Query on a Property Graph



└
:*knows*

(10,11), (11,12), (12,11),
(12,13), (15,16)

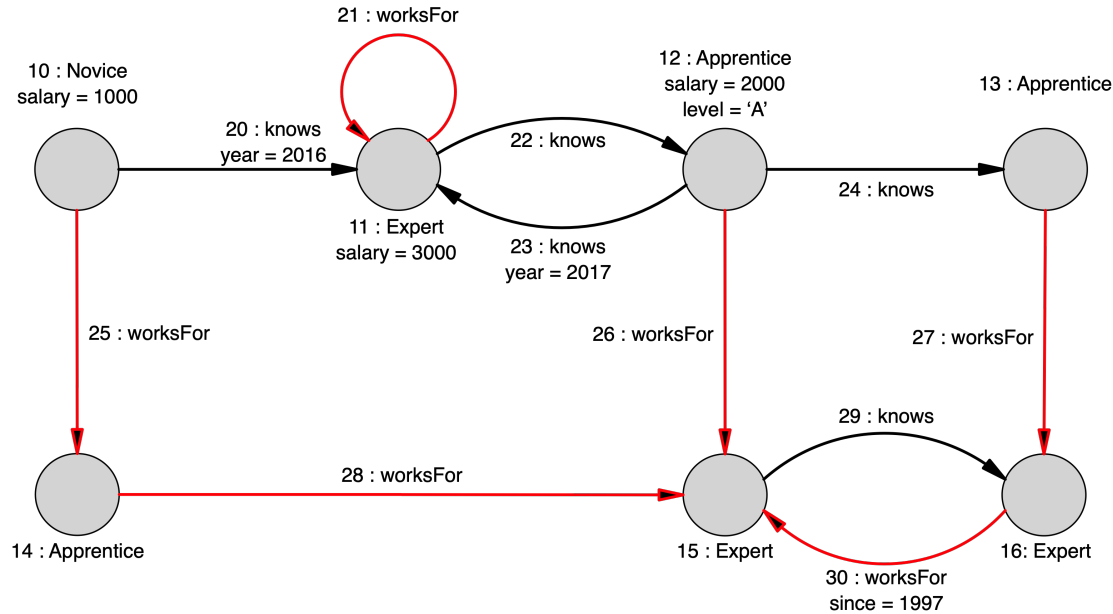
Example for Evaluating a Regular Path Query on a Property Graph



$:knows^+$

(10,11), (11,12), (12,11),
(12,13), (15,16), (10,12),
(11,11), (11,13), (12,12),
(10,13)

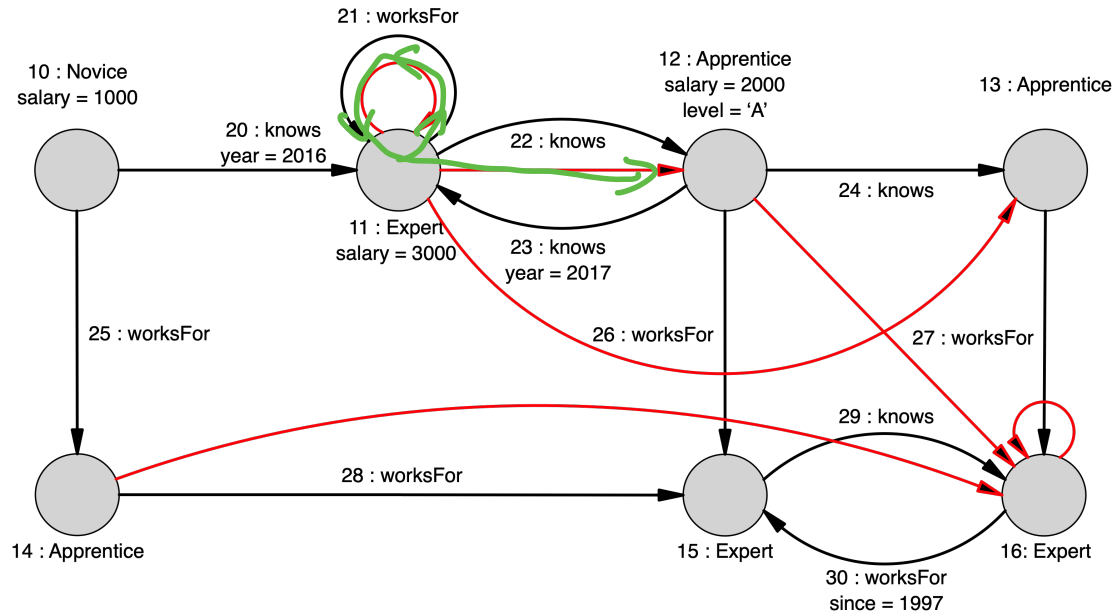
Example for Evaluating a Regular Path Query on a Property Graph



└─ :worksFor

(11,11), (10,14), (12,15),
(13, 16), (14,15), (16,15)

Example for Evaluating a Regular Path Query on a Property Graph



:worksFor

(11,11), (10,14), (12,15),
(13, 16), (14,15), (16,15)

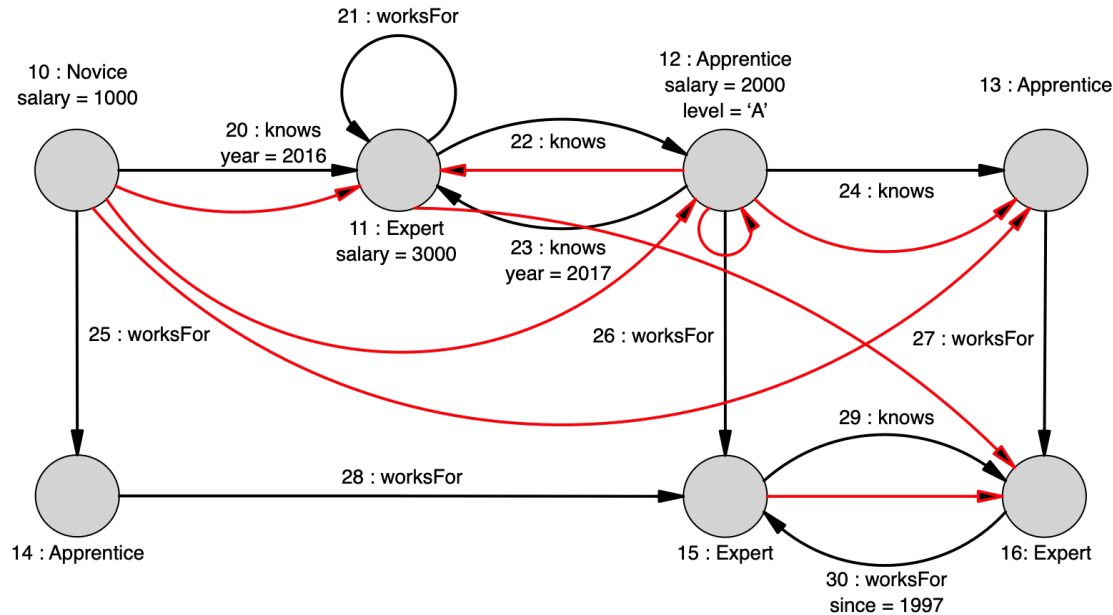
:knows⁺

(10,11), (11,12), (12,11),
(12,13), (15,16), (10,12),
(11,11), (11,13), (12,12),
(10,13)

:worksFor/ :knows⁺

(11,12), (11,11), (11,13),
(12,16), (14,16), (16,16)

Example for Evaluating a Regular Path Query on a Property Graph



:knows

(10,11), (11,12), (12,11),
(12,13), (15,16)

:worksFor/:knows⁺

(11,12), (11,11), (11,13),
(12,16), (14,16), (16,16)

:knows/:worksFor/:knows⁺

(10,12), (10,11), (10,13),
(11,16), (12,12), (12,11),
(12,13), (15,16)

can self
be ✓

Conjunctive Graph Queries

We want to identify substructures (patterns) in a graph

Conjunctive Graph Queries

We want to identify substructures (patterns) in a graph

- Example: Pairs of people in a social network who have shared connections to both a professional society and the city where they live
 - Conjunctive graph queries (CQ) enable us to query such subgraph patterns
-
- CQ asks for all subgraphs that match a given graph pattern
 - CQs are also known as “subgraph pattern matching queries”

Syntax of Conjunctive Graph Queries

Intuitively, a CQ is a set of edge predicates (edge label with a pair of vertices) that forms a subgraph pattern

Definition

Let $\mathcal{V}$ denotes a set of vertex variables. The *conjunctive graph queries* are all and only those expressions of the form

$$\underbrace{(z_1, \dots, z_m)}_{\text{out put}} \leftarrow \underbrace{a_1(x_1, y_1), \dots, a_n(x_n, y_n)}_{\text{know, works for ...}}$$

where

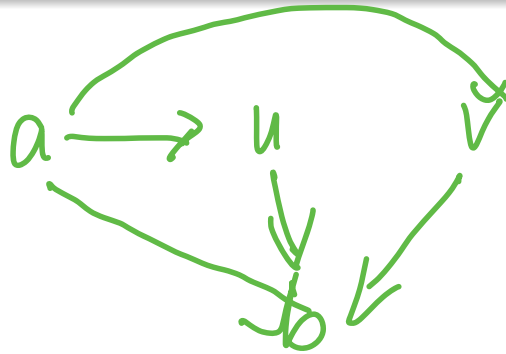
- $m \geq 0$ and $n \geq 0$,
- $x_1, y_1, \dots, x_n, y_n \in \mathcal{V}$,
- $a_1, \dots, a_n \in \mathcal{L}$, and
- for each $0 < i \leq m$, it holds that $z_i \in \{x_1, y_1, \dots, x_n, y_n\}$.

We call m the arity of the expression.

Example of a Conjunctive Graph Query

Example

- $q_1 = (a, b) \leftarrow :knows(a, b)$
- $q_2 = (a, b) \leftarrow :knows(a, u), :worksFor(a, v), :knows(v, b), :worksFor(u, b)$



Semantics of Conjunctive Graph Queries

- Key idea: bind vertex variables to vertices
- A set of variable bindings is a *valid* answer if and only if all predicates in the CQ are true on the given property graph

Definition

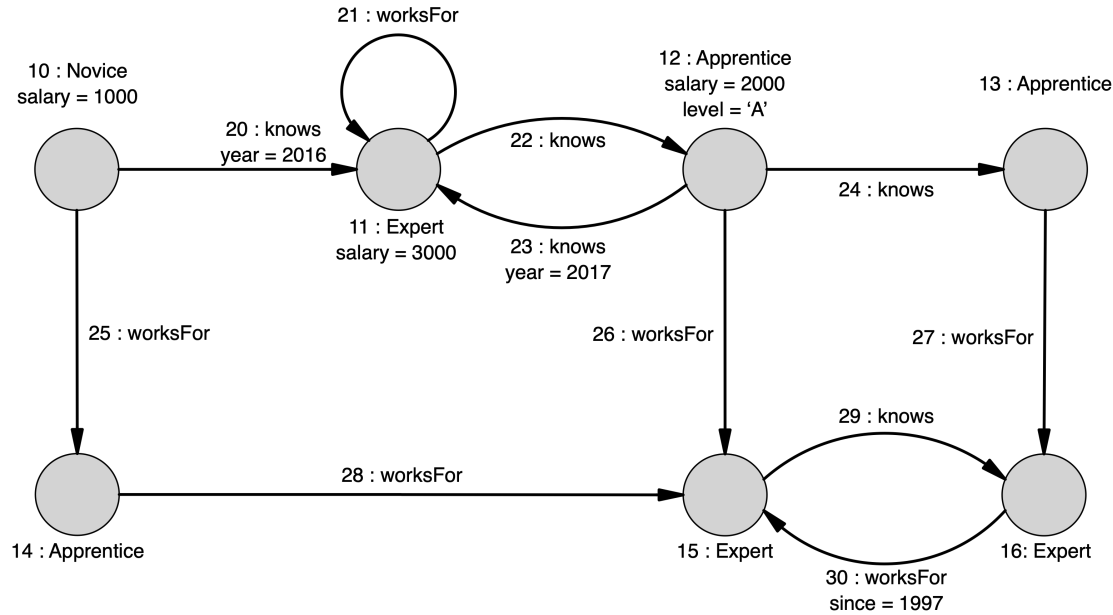
For a property graph $G = (V, E, \eta, \lambda, \nu)$ and a conjunctive graph query $q = (z_1, \dots, z_m) \leftarrow a_1(x_1, y_1), \dots, a_n(x_n, y_n)$, a *mapping* for q on G is a function $\mu : \mathcal{V} \rightarrow V$ such that for each $1 \leq i \leq n$,

- there is some edge $e_i \in E$ where
- $\eta(e_i) = (\mu(x_i), \mu(y_i))$ and
- $a_i \in \lambda(e_i)$.

The semantics of evaluating q over G is the m -ary relation $\llbracket q \rrbracket_G \subseteq V^m$ defined as

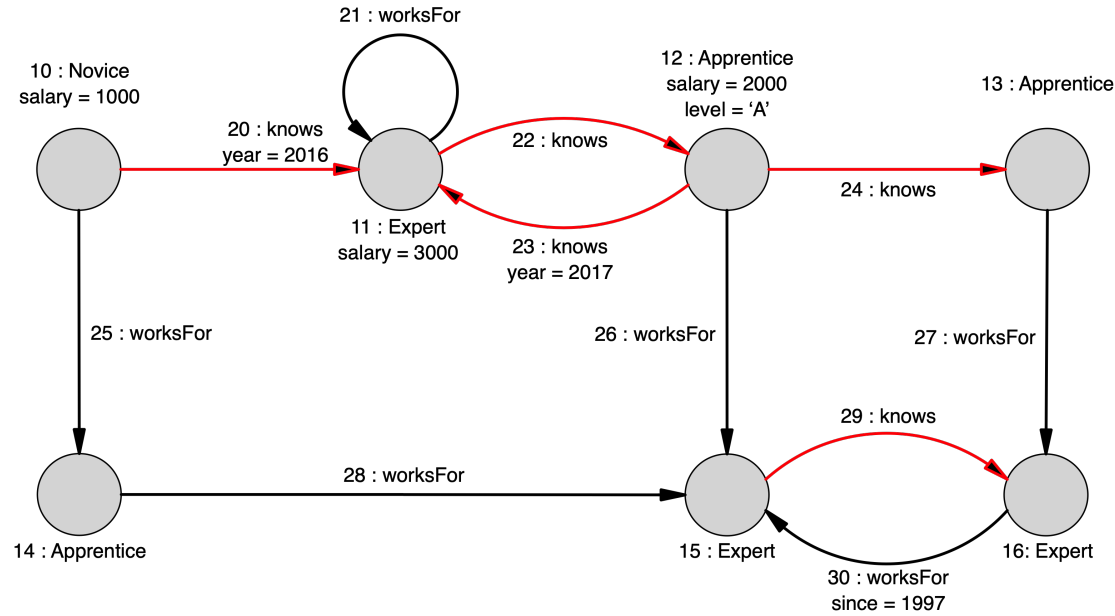
$$\llbracket q \rrbracket_G = \{(\mu(z_1), \dots, \mu(z_m)) \mid \mu \text{ is a mapping for } q \text{ on } G\}.$$

Example of the Semantics of a Conjunctive Graph Query



$q_1(a, b) \leftarrow :knows(a, b)$

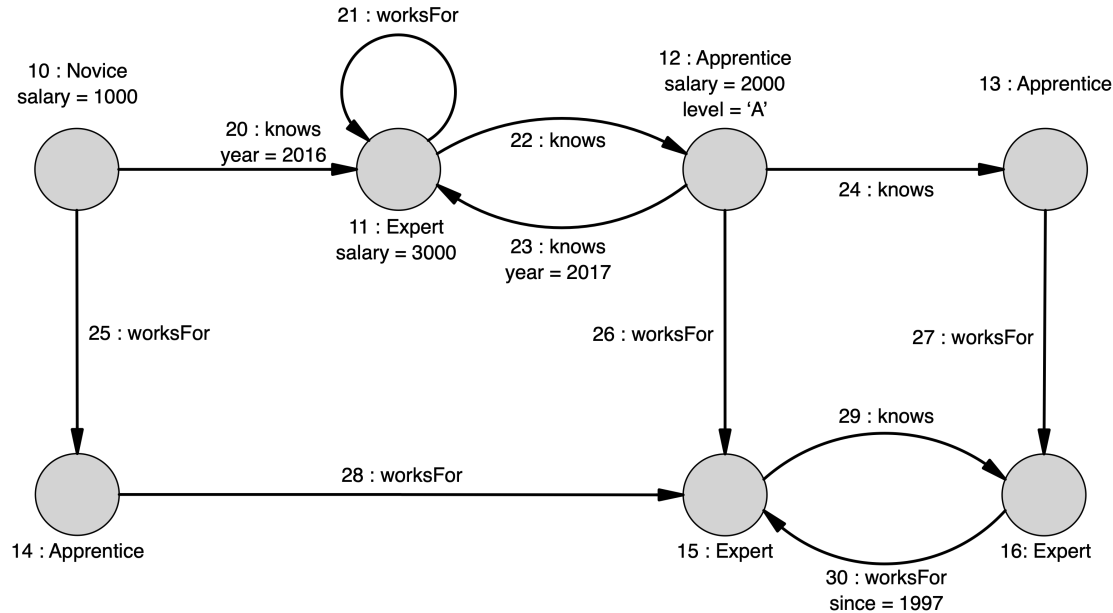
Example of the Semantics of a Conjunctive Graph Query



$q_1(a, b) \leftarrow :knows(a, b)$

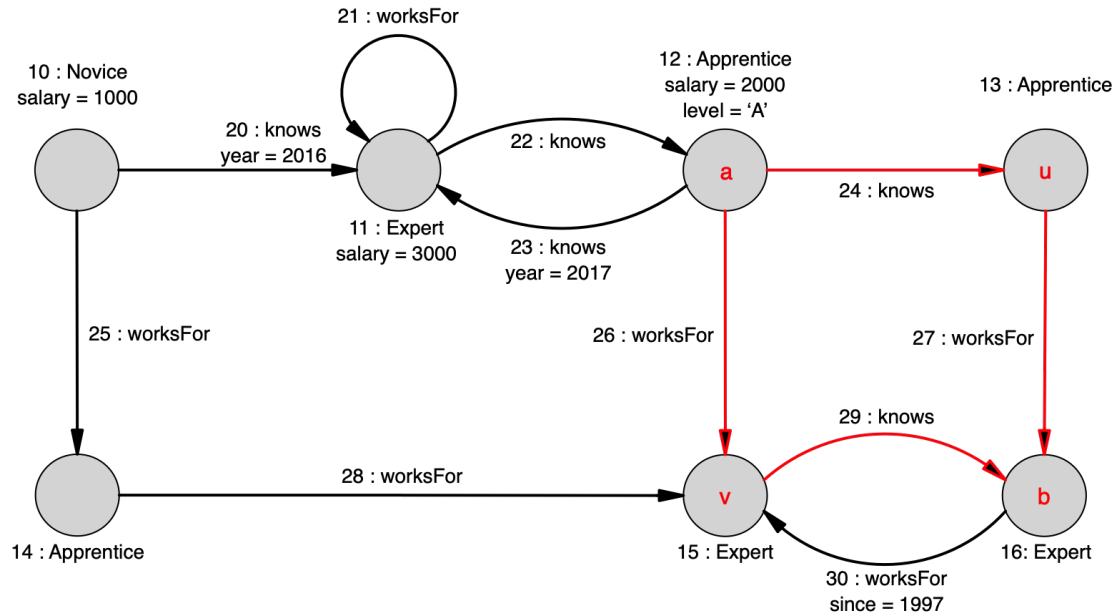
$(10,11), (11,12), (12,11),$
 $(12,13), (15,16)$

Example of the Semantics of a Conjunctive Graph Query



$q_2(a, b) \leftarrow :knows(a, u),$
 $:worksFor(a, v),$
 $:knows(v, b),$
 $:worksFor(u, b)$

Example of the Semantics of a Conjunctive Graph Query



$q_2(a, b) \leftarrow :knows(a, u),$
 $:worksFor(a, v),$
 $:knows(v, b),$
 $:worksFor(u, b)$

- $a \mapsto 12, u \mapsto 13,$
 $v \mapsto 15, b \mapsto 16$
- $:knows(12, 13),$
 $:worksFor(12, 15),$
 $:knows(15, 16),$
 $:worksFor(13, 16)$
- $q_2(12, 16)$

Conjunctive Regular Path Queries

Combine Path Regular Queries and Conjunctive Graph Queries

Definition

Let $\mathcal{V}$ denotes a set of vertex variables. The *conjunctive regular path queries* are all and only those expressions of the form

$$(z_1, \dots, z_m) \leftarrow \underbrace{\alpha_1(x_1, y_1), \dots, \alpha_n(x_n, y_n)}_{\text{regular path queries}}$$

where

- $m \geq 0$ and $n \geq 0$,
- $x_1, y_1, \dots, x_n, y_n \in \mathcal{V}$,
- $\alpha_1, \dots, \alpha_n \in RPQ$, and
- for each $0 < i \leq m$, it holds that $z_i \in \{x_1, y_1, \dots, x_n, y_n\}$.

We call m the arity of the expression.

Example of a Conjunctive Regular Path Query

Example

- $q = (a, b) \leftarrow :knows/:worksFor/:knows^+(a, b)$
- $q' = (a, b) \leftarrow :knows^+(a, u), :worksFor(u, b), :worksFor_{\mathcal{R}}^+(a, v), :knows(v, b)$

Semantics of Conjunctive Regular Path Queries

Definition

For a property graph $G = (V, E, \eta, \lambda, \nu)$ and a conjunctive regular path query

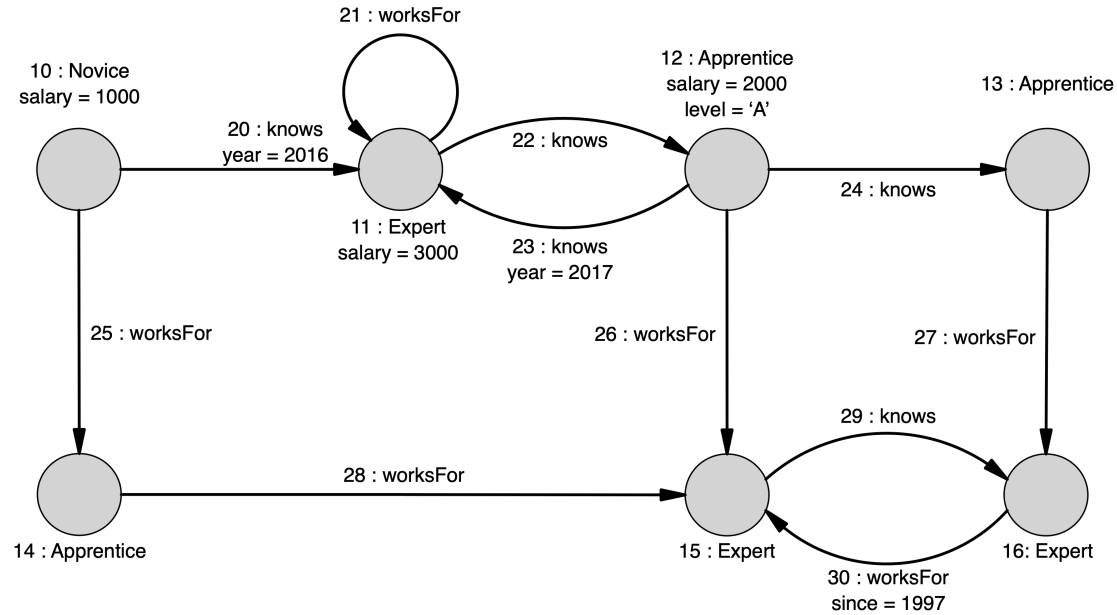
$$q = (z_1, \dots, z_m) \leftarrow \alpha_1(x_1, y_1), \dots, \alpha_n(x_n, y_n),$$

a *mapping* for q on G is a function $\mu : \mathcal{V} \rightarrow V$ such that for each $1 \leq i \leq n$, $(\mu(x_i), \mu(y_i)) \in \llbracket \alpha_i \rrbracket_G$.

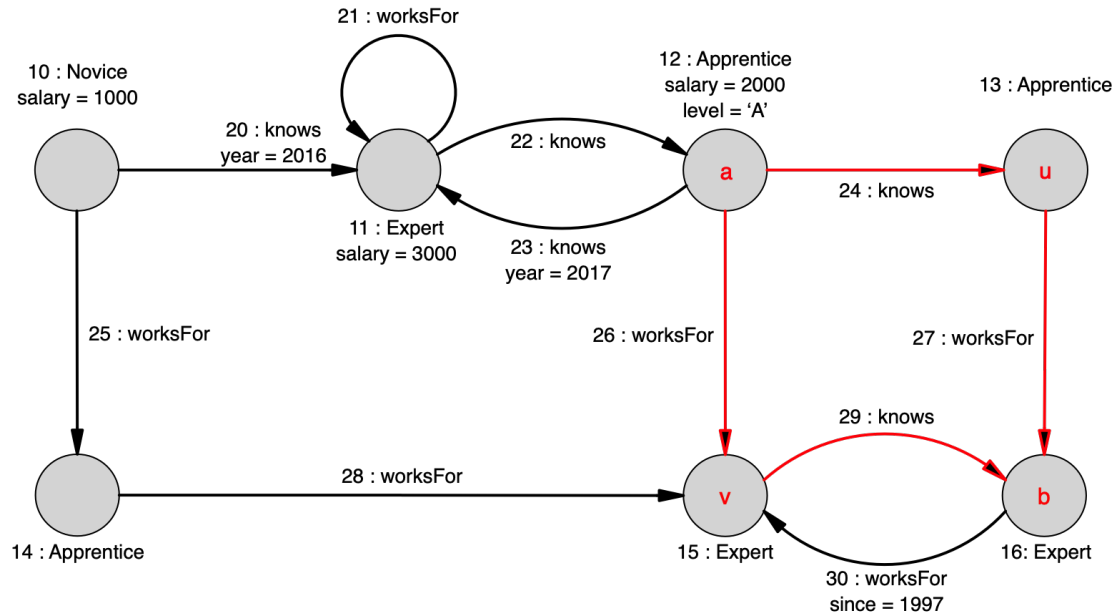
The semantics of evaluating q over G is the m -ary relation $\llbracket q \rrbracket_G \subseteq V^m$ defined as

$$\llbracket q \rrbracket_G = \{(\mu(z_1), \dots, \mu(z_m)) \mid \mu \text{ is a mapping for } q \text{ on } G\}.$$

Example of the Semantics of a Conjunctive Regular Path Query



Example of the Semantics of a Conjunctive Regular Path Query

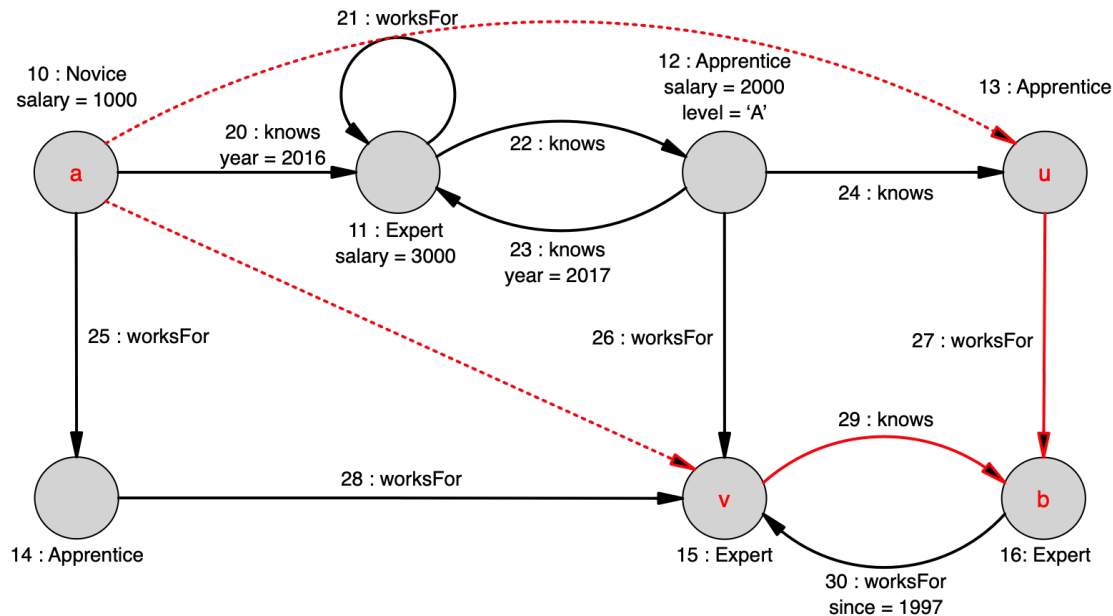


both path.

$q'(a, b) \leftarrow :knows^+(a, u),$
 $:worksFor(u, b),$
 $:worksFor^+(a, v),$
 $:knows(v, b)$

- $a \mapsto 12, u \mapsto 13,$
 $v \mapsto 15, b \mapsto 16$
- $:knows^+(12, 13),$
 $:worksFor(13, 16),$
 $:worksFor^+(12, 15),$
 $:knows(15, 16)$
- $q_2(12, 16)$

Example of the Semantics of a Conjunctive Regular Path Query



$q'(a, b) \leftarrow :knows^+(a, u),$
 $:worksFor(u, b),$
 $:worksFor^+(a, v),$
 $:knows(v, b)$

- $a \mapsto 10, u \mapsto 13,$
 $v \mapsto 15, b \mapsto 16$
- $:knows^+(10, 13),$
 $:worksFor(13, 16),$
 $:worksFor^+(10, 15),$
 $:knows(15, 16)$
- $q_2(10, 16)$

An alternative visual representation of conjunctive graph pattern queries

Definition

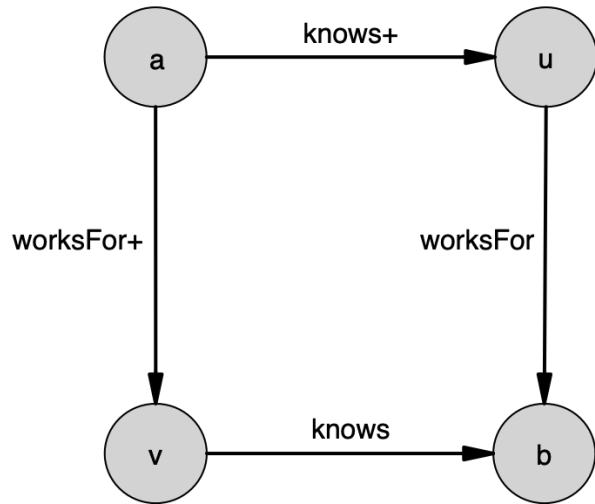
Given a CRPQ $q = (z_1, \dots, z_m) \leftarrow \alpha_1(x_1, y_1), \dots, \alpha_n(x_n, y_n)$, the *graph pattern* for q is the directed edge-labelled graph $q[z_1, \dots, z_m]$ having

- node set $\{x_1, y_1, \dots, x_n, y_n\}$, and
- for each $1 \leq i \leq n$, an edge from x_i to y_i with edge label α_i .

Given a property graph G , the semantics $\llbracket q[z_1, \dots, z_m] \rrbracket_G$ of evaluating the graph pattern $q[z_1, \dots, z_m]$ on G is the same as evaluating the CRPQ q on G , that is,

$$\llbracket q[z_1, \dots, z_m] \rrbracket_G := \llbracket q \rrbracket_G.$$

Example of a Graph Pattern



CRPQ

$$q' = (a, b) \leftarrow :knows^+(a, u), :worksFor(u, b), \\ :worksFor^+(a, v), :knows(v, b)$$

Unions of Conjunctive Regular Path Queries

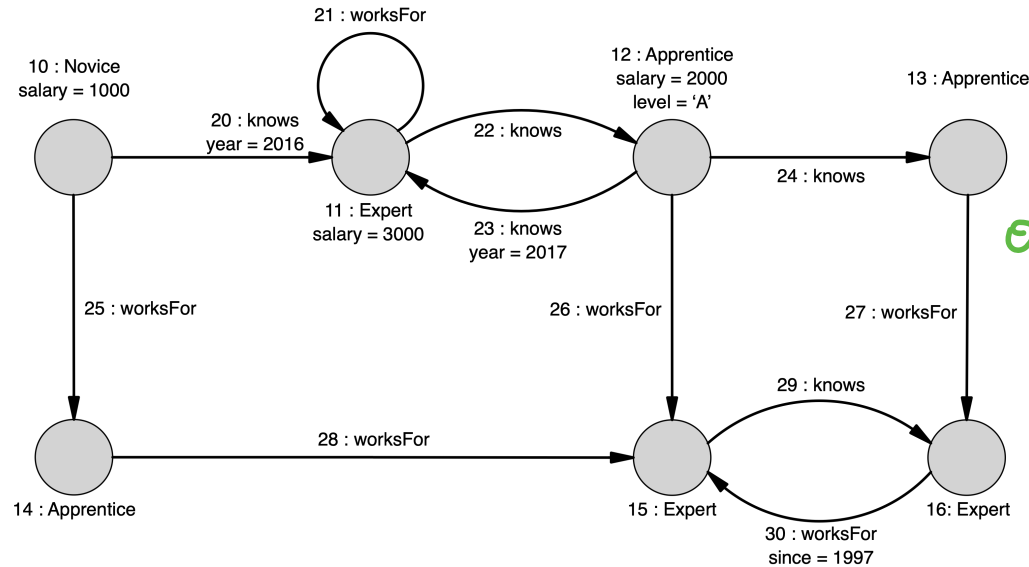
Definition

A *union of conjunctive regular path queries (UCRPQ)* is a finite non-empty set $Q \subseteq \text{CRPQ}$, each element of which has the same arity m . We also say that Q is of arity m .

For a given property graph $G = (V, E, \eta, \lambda, \nu)$ and $Q \in \text{UCRPQ}$ of arity m , the semantics of evaluating Q over G is the m -ary relation $\llbracket Q \rrbracket_G \subseteq V^m$ defined as

$$\llbracket Q \rrbracket_G := \bigcup_{q \in Q} \llbracket q \rrbracket_G .$$

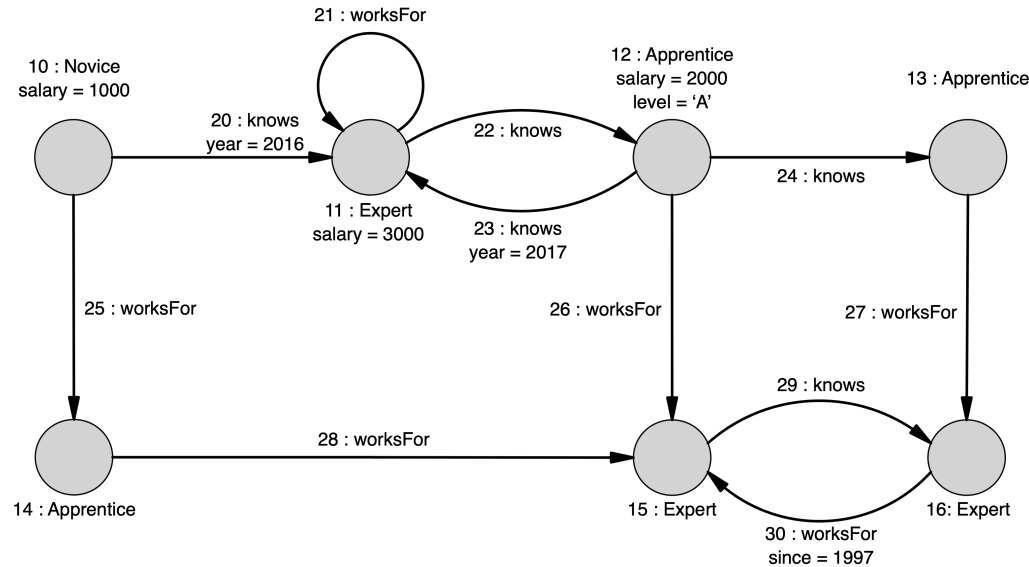
Example of a Union of Conjunctive Property Graph Query



UCRPQ Q

$(x) \leftarrow :knows/:worksFor(x, y)$
or
 $(x) \leftarrow :worksFor/:worksFor(x, y)$

Example of a Union of Conjunctive Property Graph Query



UCRPQ Q

$(x) \leftarrow :knows/:worksFor(x, y)$

$(x) \leftarrow :worksFor/:worksFor(x, y)$

Answer

10, 11, 12, 13, 15

RPQ

$$RPQ \subset CRPQ$$

$$RPQ \subset CRPQ \subset UCRPQ$$

Relationships

$$RPQ \subset CRPQ \subset UCRPQ$$

$$CQ$$

Relationships

$$RPQ \subset CRPQ \subset UCRPQ$$

$$CQ \subset CRPQ$$

Relationships

$$RPQ \subset CRPQ \subset UCRPQ$$

$$CQ \subset CRPQ$$

$$RPQ \not\subset CQ$$

Relationships

$$RPQ \subset CRPQ \subset UCRPQ$$

$$CQ \subset CRPQ$$

$$RPQ \not\subset CQ$$

$$CQ \not\subset RPQ$$

Definition

The *relational algebra* (RA) expressions are defined by the grammar of the RPQ's, extended by two additional production rules:

- $id \in RA$,
- If $e, f \in RA$, then $e - f \in RA$.

The semantics of RA expressions on property graph G extends as follows:

- If $g = id$, then $\llbracket g \rrbracket_G = \{(s, s) \mid s \in V\}$
- If $g = e - f$, then $\llbracket g \rrbracket_G = \llbracket e \rrbracket_G - \llbracket f \rrbracket_G$.

output of e with out output of f

Note that we can express intersection $e \cap f$ as $e - (e - f)$.

Example of a Relational Algebra Expression

Example

$q_1 = (:worksFor^+ \cap :knows^+) - worksFor$ retrieves all pairs of people (x, y) such that x knows y and x indirectly works for y

Example of a Relational Algebra Expression

Example

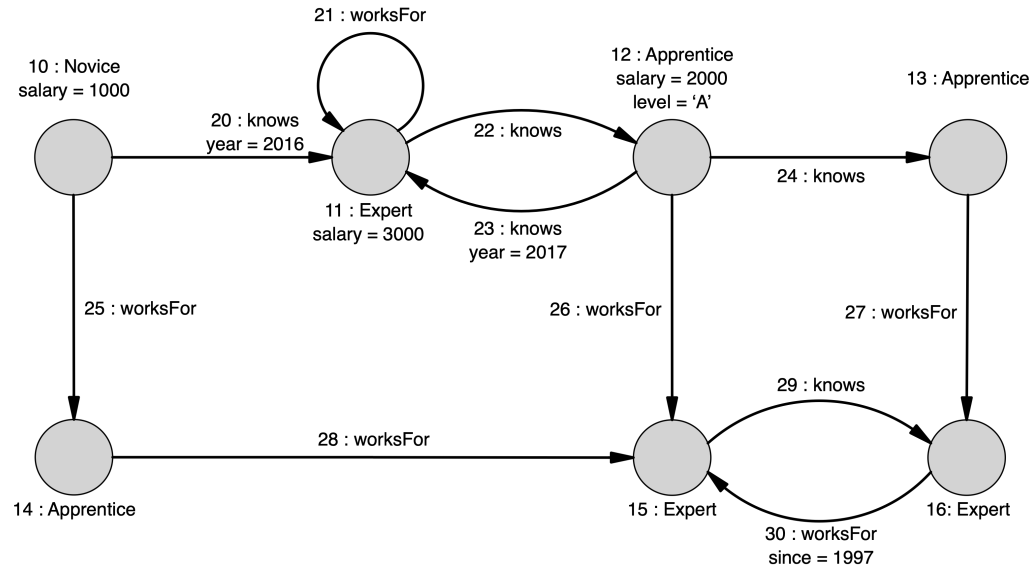
$q_1 = (:worksFor^+ \cap :knows^+) - worksFor$ retrieves all pairs of people (x, y) such that x knows y and x indirectly works for y

Example

$q_2 = (:worksFor \cap :id) / :knows$ finds all pairs of people (x, y) where x is a person who works for themselves and y is someone x knows

not same
↗

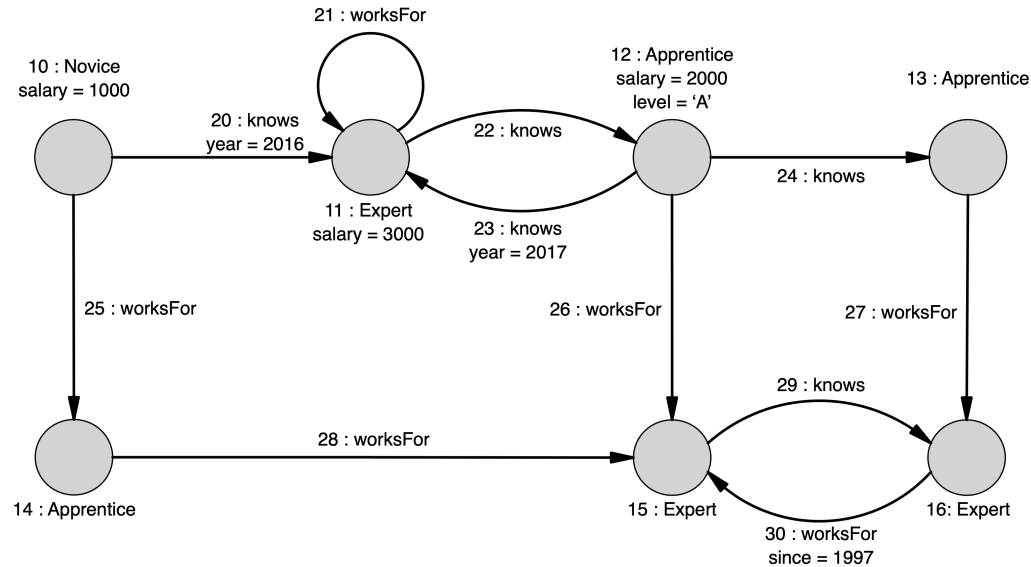
Example of Evaluating an RA Expression



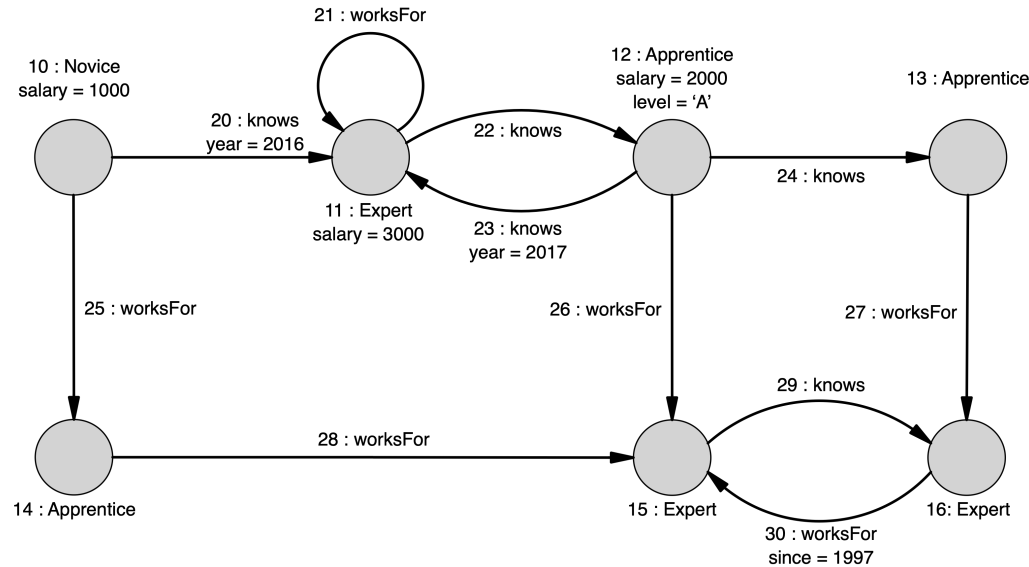
$(:worksFor^+ \cap :knows^+) - worksFor$

$(:worksFor \cap :id) / :knows$

Example of Evaluating an RA Expression


$$(:worksFor^+ \cap :knows^+) - worksFor$$
$$\emptyset$$
$$(:worksFor \cap :id) / :knows$$

Example of Evaluating an RA Expression


$$(:worksFor^+ \cap :knows^+) - worksFor$$
$$\emptyset$$
$$(:worksFor \cap :id) / :knows$$
$$(11, 12)$$

Regular Property Graph Queries

So far, our queries do not support reasoning about the features of the property graph model, including the property values of nodes and edges,

- Introduce *Regular Property Graph Queries* to generalize previous queries
- RPGQs are well-behaved as their containment problem remains decidable

Two formalisms

- Regular property graph logic
- Regular property graph algebra
- Both formalisms are equivalent

Regular Property Graph Logic (RPGLog)

A fragment of standard non-recursive Datalog tailored to property graphs

Regular Property Graph Logic (RPGLog)

A fragment of standard non-recursive Datalog tailored to property graphs

- RPGLog corresponds to unions of conjunctive queries on graphs
- That is, all predicates are binary or unary, corresponding to edge labels and node labels, respectively $= \tilde{v}$ $-\tilde{v}$
- They are augmented with edge variables and a transitive closure operation “*”
- RPGLog queries can have nesting of subqueries

Edge and Vertex variables

- $\mathcal{E}$ a set of edge labels, $\mathcal{V}$ a set of vertex labels, $\mathcal{E} \cap \mathcal{V} = \emptyset$ 不能重复.
- Main role:
Reference to edges and vertices in head and body predicates of rules to filter on their properties

Edge and Vertex variables

- $\mathcal{E}$ a set of edge labels, $\mathcal{V}$ a set of vertex labels, $\mathcal{E} \cap \mathcal{V} = \emptyset$
- Main role:
Reference to edges and vertices in head and body predicates of rules to filter on their properties

Context identifiers

- $\mathcal{C}$ denotes a set of context identifiers
- Main role: Distinguish edges created by different rules

output

edges

propag, value...

$head \leftarrow body_1, \dots, body_n, \text{constraint}_1, \dots, \text{constraint}_m$ with $n > 0$ and $m \geq 0$

$head \leftarrow body_1, \dots, body_n, constraint_1, \dots, constraint_m$ with $n > 0$ and $m \geq 0$

Body predicates $body_i$

- Each body predicate $body_i$ is of one of the following forms:
 - $p(x, y) \text{ AS } e$
 - $p^*(x, y)$
 - $p(x)$

where $p \in \mathcal{L}$, $x, y \in \mathcal{V}$, and $e \in \mathcal{E}$

$head \leftarrow body_1, \dots, body_n, constraint_1, \dots, constraint_m$ with $n > 0$ and $m \geq 0$

Body predicates $constraint_i$

- Each body predicate $constraint_i$ is of one of the following forms:

- $x.p \theta y.q$, or *compare*
- $x.p \theta val$

where $x, y \in \mathcal{V} \cup \mathcal{E}$, both x and y appear in a non-constraint body predicate (not necessarily the same predicate), $p, q \in \mathcal{K}$, $val \in \mathcal{N}$ and $\theta \in \{=, \neq, <, >, \leq, \geq\}$, or, of the form

- $x = y$

where $x, y \in \mathcal{V}$ and both x and y appear in a non-constraint body predicate (not necessarily the same predicate)

$head \leftarrow body_1, \dots, body_n, constraint_1, \dots, constraint_m$ with $n > 0$ and $m \geq 0$

Heads

- the head predicate $head$ is either of the form

$$p(x, y) \text{ IN } c,$$

where $p \in \mathcal{L}$, $c \in \mathcal{C}$, and $x, y \in \mathcal{V}$, and both x and y appear in a body predicate (not necessarily the same predicate), or

- of the form

$$result(x_1, \dots, x_n),$$

need same shape

where $n \geq 0$, $result$ is a reserved predicate not in $\mathcal{L}$, $x_1, \dots, x_n \in \mathcal{V}$, and each x_i appears in a body predicate.

$head \leftarrow body_1, \dots, body_n, constraint_1, \dots, constraint_m$ with $n > 0$ and $m \geq 0$

Dependency Graph

- Given a set R of rules, the *dependency graph* of R is the directed graph
- Having as node set the elements of $\mathcal{L}$ appearing in a predicate of a rule of R
- With an edge from x to y iff x appears in the head of a rule r and y appears in the body of r
- R is *recursive* if the dependency graph of R has a cycle, otherwise R is *non-recursive*

$$head \leftarrow body_1, \dots, body_n, constraint_1, \dots, constraint_m \text{ with } n > 0 \text{ and } m \geq 0$$

RPGLog query

A finite non-empty non-recursive set of rules such that at least one rule has head predicate *result* and all *result* predicates have the same arity

Example for an RPGLog

Example

edge *lab*

$:knownExpert(y, y) \text{ IN } a \leftarrow :knows(x, y) \text{ AS } k, :Expert(z), k.year > 2000, y = z.$
 $:related(x, y) \text{ IN } b \leftarrow :knows(x, y). \quad \rightarrow \text{滿足之算 related}$
 $:related(x, y) \text{ IN } b \leftarrow :worksFor(x, y).$
 $result(x) \leftarrow :knownExpert(x, x), :related^*(x, y),$
 $x.salary < 5000, y.salary < x.salary.$

Example for an RPGLog

Example

```
:knownExpert(y, y) IN a    $\leftarrow$  :knows(x, y) AS k, :Expert(z), k.year > 2000, y = z.  
  :related(x, y) IN b      $\leftarrow$  :knows(x, y).  
  :related(x, y) IN b      $\leftarrow$  :worksFor(x, y).  
    result(x)             $\leftarrow$  :knownExpert(x, x), :related*(x, y),  
                                x.salary < 5000, y.salary < x.salary.
```

retrieves the experts known after the year 2000, with a salary below \$5,000, and related via knowing or working relationships to someone with a lower salary

Example for an RPGLog

Example

$:knownExpert(y, y) \text{ IN } a \leftarrow :knows(x, y) \text{ AS } k, :Expert(z), k.year > 2000, y = z.$
 $:related(x, y) \text{ IN } b \leftarrow :knows(x, y).$
 $:related(x, y) \text{ IN } b \leftarrow :worksFor(x, y).$
 $result(x) \leftarrow :knownExpert(x, x), :related^*(x, y),$
 $x.salary < 5000, y.salary < x.salary.$

Notes

- We omit “AS e ” from “ $p(x, y) \text{ AS } e$ ” in a rule body if e does not appear elsewhere in that rule body
- The query can be simplified, e.g. by dropping the conjunct $y = z$ and replacing z by y in the body of the first rule

Example for an RPGLog

Example

```
:knownExpert(y, y) IN a ← :knows(x, y) AS k, :Expert(z), k.year > 2000, y = z.  
  :related(x, y) IN b ← :knows(x, y).  
  :related(x, y) IN b ← :worksFor(x, y).  
    result(x) ← :knownExpert(x, x), :related*(x, y),  
               x.salary < 5000, y.salary < x.salary.
```

Notes

- Since both rules with head *:related*(x, y) IN *b* share the same context identifier, only one edge *:related* is derived for a particular vertex pair (x, y), even if both rules apply to x and y
- If both rules declared a different context identifier (eg. one uses *b* and the other *c*), then the query would derive two distinct *:related* edges for (x, y)

$G = (V, E, \eta, \lambda, \nu)$ property graph, and $q \in \text{RPGLog}$, $r \in q$:

$head \leftarrow body_1, \dots, body_n, constraint_1, \dots, constraint_m$ with $n > 0$ and $m \geq 0$

$G = (V, E, \eta, \lambda, \nu)$ property graph, and $q \in \text{RPGLog}$, $r \in q$:

$\text{head} \leftarrow \text{body}_1, \dots, \text{body}_n, \text{constraint}_1, \dots, \text{constraint}_m$ with $n > 0$ and $m \geq 0$

Mapping

- A function $\mu : \mathcal{V} \cup \mathcal{E} \rightarrow \mathcal{O}$ assigning object identifiers to variables
- A mapping μ *satisfies* r in G if and only if μ satisfies each body predicate b of r

$G = (V, E, \eta, \lambda, \nu)$ property graph, and $q \in \text{RPGLog}$, $r \in q$:

$\text{head} \leftarrow \text{body}_1, \dots, \text{body}_n, \text{constraint}_1, \dots, \text{constraint}_m$ with $n > 0$ and $m \geq 0$

Satisfaction of a predicate b in G

- If $b = "p(x, y) \text{ AS } e"$:
 - in the case where $p \in \lambda(E)$, then it holds that there is some $\text{edge} \in E$ such that $\mu(e) = \text{edge}$, $p \in \lambda(\text{edge})$, and $\eta(\text{edge}) = (\mu(x), \mu(y))$, and
 - in the case where $p \notin \lambda(E) \cup \lambda(V)$, then it holds that there is some $\text{edge} \in \llbracket p \rrbracket_G$ such that $(\mu(x), \mu(y)) \in \text{edge}$, and $\mu(e) = \text{edge}$.

$G = (V, E, \eta, \lambda, \nu)$ property graph, and $q \in \text{RPGLog}$, $r \in q$:

$\text{head} \leftarrow \text{body}_1, \dots, \text{body}_n, \text{constraint}_1, \dots, \text{constraint}_m$ with $n > 0$ and $m \geq 0$

Satisfaction of a predicate b in G

- If $b = "p(x, y) \text{ AS } e"$:
 - in the case where $p \in \lambda(E)$, then it holds that there is some $\text{edge} \in E$ such that $\mu(e) = \text{edge}$, $p \in \lambda(\text{edge})$, and $\eta(\text{edge}) = (\mu(x), \mu(y))$, and
 - in the case where $p \notin \lambda(E) \cup \lambda(V)$, then it holds that there is some $\text{edge} \in \llbracket p \rrbracket_G$ such that $(\mu(x), \mu(y)) \in \text{edge}$, and $\mu(e) = \text{edge}$.
- If $b = "p^*(x, y)"$ where $p \notin \lambda(V)$, then it holds that $(\mu(x), \mu(y)) \in \llbracket p^* \rrbracket_G$.

$G = (V, E, \eta, \lambda, \nu)$ property graph, and $q \in \text{RPGLog}$, $r \in q$:

$\text{head} \leftarrow \text{body}_1, \dots, \text{body}_n, \text{constraint}_1, \dots, \text{constraint}_m$ with $n > 0$ and $m \geq 0$

Satisfaction of a predicate b in G

- If $b = "p(x, y) \text{ AS } e"$:
 - in the case where $p \in \lambda(E)$, then it holds that there is some $\text{edge} \in E$ such that $\mu(e) = \text{edge}$, $p \in \lambda(\text{edge})$, and $\eta(\text{edge}) = (\mu(x), \mu(y))$, and
 - in the case where $p \notin \lambda(E) \cup \lambda(V)$, then it holds that there is some $\text{edge} \in \llbracket p \rrbracket_G$ such that $(\mu(x), \mu(y)) \in \text{edge}$, and $\mu(e) = \text{edge}$.
- If $b = "p^*(x, y)"$ where $p \notin \lambda(V)$, then it holds that $(\mu(x), \mu(y)) \in \llbracket p^* \rrbracket_G$.
- If $b = "p(x)"$ where $p \in \lambda(V)$, then it holds that $\mu(x) \in V$ and $p \in \lambda(\mu(x))$.

$G = (V, E, \eta, \lambda, \nu)$ property graph, and $q \in \text{RPGLog}$, $r \in q$:

$\text{head} \leftarrow \text{body}_1, \dots, \text{body}_n, \text{constraint}_1, \dots, \text{constraint}_m$ with $n > 0$ and $m \geq 0$

Satisfaction of a predicate b in G

- If $b = "p(x, y) \text{ AS } e"$:
 - in the case where $p \in \lambda(E)$, then it holds that there is some $\text{edge} \in E$ such that $\mu(e) = \text{edge}$, $p \in \lambda(\text{edge})$, and $\eta(\text{edge}) = (\mu(x), \mu(y))$, and
 - in the case where $p \notin \lambda(E) \cup \lambda(V)$, then it holds that there is some $\text{edge} \in \llbracket p \rrbracket_G$ such that $(\mu(x), \mu(y)) \in \text{edge}$, and $\mu(e) = \text{edge}$.
- If $b = "p^*(x, y)"$ where $p \notin \lambda(V)$, then it holds that $(\mu(x), \mu(y)) \in \llbracket p^* \rrbracket_G$.
- If $b = "p(x)"$ where $p \in \lambda(V)$, then it holds that $\mu(x) \in V$ and $p \in \lambda(\mu(x))$.
- If $b = "x.p \theta y.q"$, then it holds that $\nu(\mu(x), p) \theta \nu(\mu(y), q)$.

$G = (V, E, \eta, \lambda, \nu)$ property graph, and $q \in \text{RPGLog}$, $r \in q$:

$\text{head} \leftarrow \text{body}_1, \dots, \text{body}_n, \text{constraint}_1, \dots, \text{constraint}_m$ with $n > 0$ and $m \geq 0$

Satisfaction of a predicate b in G

- If $b = "p(x, y) \text{ AS } e"$:
 - in the case where $p \in \lambda(E)$, then it holds that there is some $\text{edge} \in E$ such that $\mu(e) = \text{edge}$, $p \in \lambda(\text{edge})$, and $\eta(\text{edge}) = (\mu(x), \mu(y))$, and
 - in the case where $p \notin \lambda(E) \cup \lambda(V)$, then it holds that there is some $\text{edge} \in \llbracket p \rrbracket_G$ such that $(\mu(x), \mu(y)) \in \text{edge}$, and $\mu(e) = \text{edge}$.
- If $b = "p^*(x, y)"$ where $p \notin \lambda(V)$, then it holds that $(\mu(x), \mu(y)) \in \llbracket p^* \rrbracket_G$.
- If $b = "p(x)"$ where $p \in \lambda(V)$, then it holds that $\mu(x) \in V$ and $p \in \lambda(\mu(x))$.
- If $b = "x.p \theta y.q"$, then it holds that $\nu(\mu(x), p) \theta \nu(\mu(y), q)$.
- If $b = "x.p \theta \text{val}"$, then it holds that $\nu(\mu(x), p) \theta \text{val}$.

$G = (V, E, \eta, \lambda, \nu)$ property graph, and $q \in \text{RPGLog}$, $r \in q$:

$\text{head} \leftarrow \text{body}_1, \dots, \text{body}_n, \text{constraint}_1, \dots, \text{constraint}_m$ with $n > 0$ and $m \geq 0$

Satisfaction of a predicate b in G

- If $b = "p(x, y) \text{ AS } e"$:
 - in the case where $p \in \lambda(E)$, then it holds that there is some $\text{edge} \in E$ such that $\mu(e) = \text{edge}$, $p \in \lambda(\text{edge})$, and $\eta(\text{edge}) = (\mu(x), \mu(y))$, and
 - in the case where $p \notin \lambda(E) \cup \lambda(V)$, then it holds that there is some $\text{edge} \in \llbracket p \rrbracket_G$ such that $(\mu(x), \mu(y)) \in \text{edge}$, and $\mu(e) = \text{edge}$.
- If $b = "p^*(x, y)"$ where $p \notin \lambda(V)$, then it holds that $(\mu(x), \mu(y)) \in \llbracket p^* \rrbracket_G$.
- If $b = "p(x)"$ where $p \in \lambda(V)$, then it holds that $\mu(x) \in V$ and $p \in \lambda(\mu(x))$.
- If $b = "x.p \theta y.q"$, then it holds that $\nu(\mu(x), p) \theta \nu(\mu(y), q)$.
- If $b = "x.p \theta \text{val}"$, then it holds that $\nu(\mu(x), p) \theta \text{val}$.
- If $b = "x = y"$, then it holds that $\mu(x) = \mu(y)$.

$G = (V, E, \eta, \lambda, \nu)$ property graph, and $q \in \text{RPGLog}$, $r \in q$:

$\text{head} \leftarrow \text{body}_1, \dots, \text{body}_n, \text{constraint}_1, \dots, \text{constraint}_m$ with $n > 0$ and $m \geq 0$

Satisfaction of a predicate b in G

- If $b = "p(x, y) \text{ AS } e"$:
 - in the case where $p \in \lambda(E)$, then it holds that there is some $\text{edge} \in E$ such that $\mu(e) = \text{edge}$, $p \in \lambda(\text{edge})$, and $\eta(\text{edge}) = (\mu(x), \mu(y))$, and
 - in the case where $p \notin \lambda(E) \cup \lambda(V)$, then it holds that there is some $\text{edge} \in \llbracket p \rrbracket_G$ such that $(\mu(x), \mu(y)) \in \text{edge}$, and $\mu(e) = \text{edge}$.
- If $b = "p^*(x, y)"$ where $p \notin \lambda(V)$, then it holds that $(\mu(x), \mu(y)) \in \llbracket p^* \rrbracket_G$.
- If $b = "p(x)"$ where $p \in \lambda(V)$, then it holds that $\mu(x) \in V$ and $p \in \lambda(\mu(x))$.
- If $b = "x.p \theta y.q"$, then it holds that $\nu(\mu(x), p) \theta \nu(\mu(y), q)$.
- If $b = "x.p \theta \text{val}"$, then it holds that $\nu(\mu(x), p) \theta \text{val}$.
- If $b = "x = y"$, then it holds that $\mu(x) = \mu(y)$.
- In all other cases, b is not satisfiable.

Fresh Object Identifiers

- $\omega_G : \mathcal{O} \times \mathcal{O} \times \mathcal{C} \rightarrow \mathcal{O} \setminus (V \cup E)$ is an injective function
- It assigns object identifiers in $\mathcal{O}$ that are distinct from the object identifiers used in G and on which ν is nowhere defined

Rule Evaluation

For $p \notin \lambda(E)$ let $\hat{p} \subseteq q$ be the possibly empty set of all rules with head predicate p

Rule Evaluation

For $p \notin \lambda(E)$ let $\hat{p} \subseteq q$ be the possibly empty set of all rules with head predicate p

$$\llbracket p \rrbracket_G = \bigcup_{r \in \hat{p}} \llbracket r \rrbracket_G$$

Rule Evaluation

For $p \notin \lambda(E)$ let $\hat{p} \subseteq q$ be the possibly empty set of all rules with head predicate p

$$\llbracket p \rrbracket_G = \bigcup_{r \in \hat{p}} \llbracket r \rrbracket_G$$

- Assume output variables (x, y) and context identifier c for the head of rule r
- $\llbracket r \rrbracket_G = \{ \{ (\mu(x), \mu(y)), \omega_G(\mu(x), \mu(y), c) \} \mid \mu \text{ is a mapping satisfying } r \text{ in } G \}$

Rule Evaluation

For $p \notin \lambda(E)$ let $\hat{p} \subseteq q$ be the possibly empty set of all rules with head predicate p

$$\llbracket p \rrbracket_G = \bigcup_{r \in \hat{p}} \llbracket r \rrbracket_G$$

- Assume output variables (x, y) and context identifier c for the head of rule r
- $\llbracket r \rrbracket_G = \{ \{ (\mu(x), \mu(y)), \omega_G(\mu(x), \mu(y), c) \} \mid \mu \text{ is a mapping satisfying } r \text{ in } G \}$

As p is non-recursively defined and G is finite, $\llbracket p \rrbracket_G$ is well-defined and finite

Transitive Closure

$$p^*(x, y)$$

Case $p \in \lambda(E)$:

$\llbracket p^* \rrbracket_G = \{(x, x) \mid x \in V\} \cup \{(x, y) \mid x, y \in V \text{ and } x \rightarrow_{p,G} y\}$ where $x \rightarrow_{p,G} y$ denotes that for some $n > 0$ there is some $e_1, \dots, e_n \in E$ and $v_1, \dots, v_{n+1} \in V$ such that $x = v_1, y = v_{n+1}$ and for $1 \leq i \leq n$, $\lambda(e_i) = p$ and $\eta(e_i) = (v_i, v_{i+1})$

Transitive Closure

$$p^*(x, y)$$

Case $p \in \lambda(E)$:

$\llbracket p^* \rrbracket_G = \{(x, x) \mid x \in V\} \cup \{(x, y) \mid x, y \in V \text{ and } x \rightarrow_{p, G} y\}$ where $x \rightarrow_{p, G} y$ denotes that for some $n > 0$ there is some $e_1, \dots, e_n \in E$ and $v_1, \dots, v_{n+1} \in V$ such that $x = v_1, y = v_{n+1}$ and for $1 \leq i \leq n$, $\lambda(e_i) = p$ and $\eta(e_i) = (v_i, v_{i+1})$

Case $p \notin \lambda(E)$:

$\llbracket p^* \rrbracket_G = \{(x, x) \mid x \in V\} \cup \{(x, y) \mid x, y \in V \text{ and } x \rightarrow_{\llbracket p \rrbracket_G} y\}$ where $x \rightarrow_{\llbracket p \rrbracket_G} y$ denotes that for some $n > 0$ there is some $e_1, \dots, e_n \in \llbracket p \rrbracket_G$ and $v_1, \dots, v_{n+1} \in V$ such that $x = v_1, y = v_{n+1}$ and for $1 \leq i \leq n$, $(v_i, v_{i+1}) \in e_i$

Transitive Closure

$$p^*(x, y)$$

Case $p \in \lambda(E)$:

$\llbracket p^* \rrbracket_G = \{(x, x) \mid x \in V\} \cup \{(x, y) \mid x, y \in V \text{ and } x \rightarrow_{p, G} y\}$ where $x \rightarrow_{p, G} y$ denotes that for some $n > 0$ there is some $e_1, \dots, e_n \in E$ and $v_1, \dots, v_{n+1} \in V$ such that $x = v_1, y = v_{n+1}$ and for $1 \leq i \leq n$, $\lambda(e_i) = p$ and $\eta(e_i) = (v_i, v_{i+1})$

Case $p \notin \lambda(E)$:

$\llbracket p^* \rrbracket_G = \{(x, x) \mid x \in V\} \cup \{(x, y) \mid x, y \in V \text{ and } x \rightarrow_{\llbracket p \rrbracket_G} y\}$ where $x \rightarrow_{\llbracket p \rrbracket_G} y$ denotes that for some $n > 0$ there is some $e_1, \dots, e_n \in \llbracket p \rrbracket_G$ and $v_1, \dots, v_{n+1} \in V$ such that $x = v_1, y = v_{n+1}$ and for $1 \leq i \leq n$, $(v_i, v_{i+1}) \in e_i$

p is non-recursively defined and G is finite, so $\llbracket p^* \rrbracket_G$ is well-defined and finite

Semantics of Evaluating an RGQLog Query on a Property Graph

Given

- G property graph,
- $q \in \text{RPGLog}$,
- $\{r_1, \dots, r_k\} \subseteq q$ the set of all rules with head predicate $\text{result}(x_1, \dots, x_n)$

Semantics of Evaluating an RGQLog Query on a Property Graph

Given

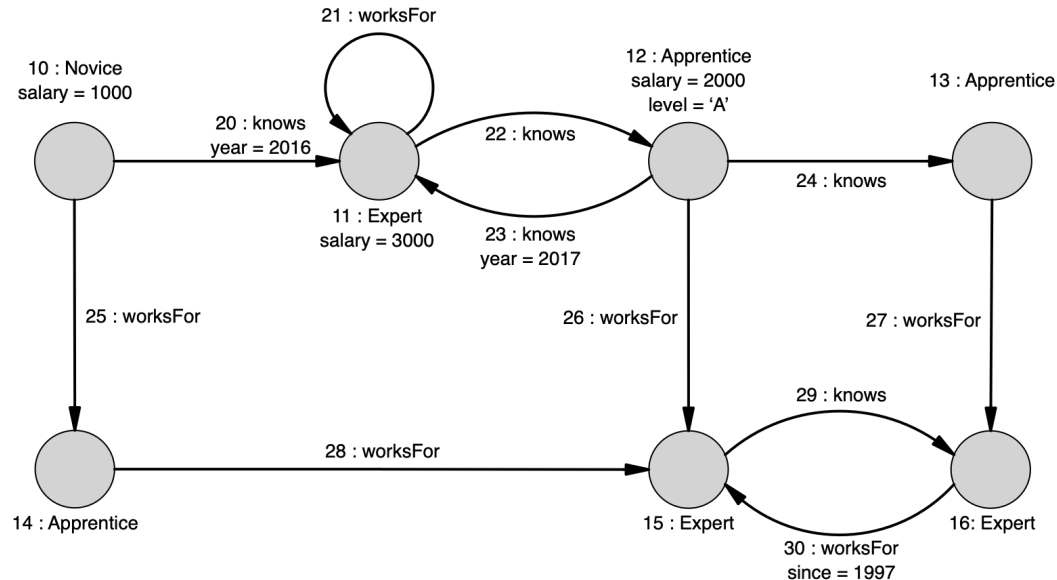
- G property graph,
- $q \in \text{RPGLog}$,
- $\{r_1, \dots, r_k\} \subseteq q$ the set of all rules with head predicate $\text{result}(x_1, \dots, x_n)$

Evaluating q on G

$$\llbracket q \rrbracket_G = \bigcup_{r \in \{r_1, \dots, r_k\}} \{(\mu(x_1), \dots, \mu(x_n)) \mid \mu \text{ is a mapping satisfying } r \text{ in } G\}$$

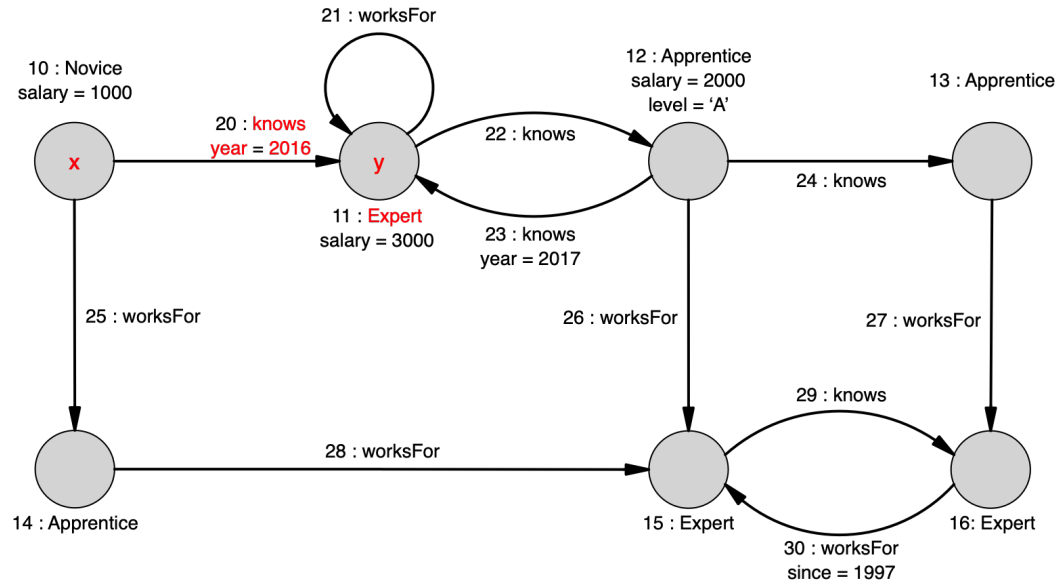
Example for RPGLog Evaluation

$:knownExpert(y, y)$ IN a $\leftarrow$ $:knows(x, y)$ AS $k, :Expert(z), k.year > 2000, y = z$.
 $:related(x, y)$ IN b $\leftarrow$ $:knows(x, y)$.
 $:related(x, y)$ IN b $\leftarrow$ $:worksFor(x, y)$.
 $result(x)$ $\leftarrow$ $:knownExpert(x, x), :related^*(x, y),$
 $x.salary < 5000, y.salary < x.salary$.



Example for RPGLog Evaluation

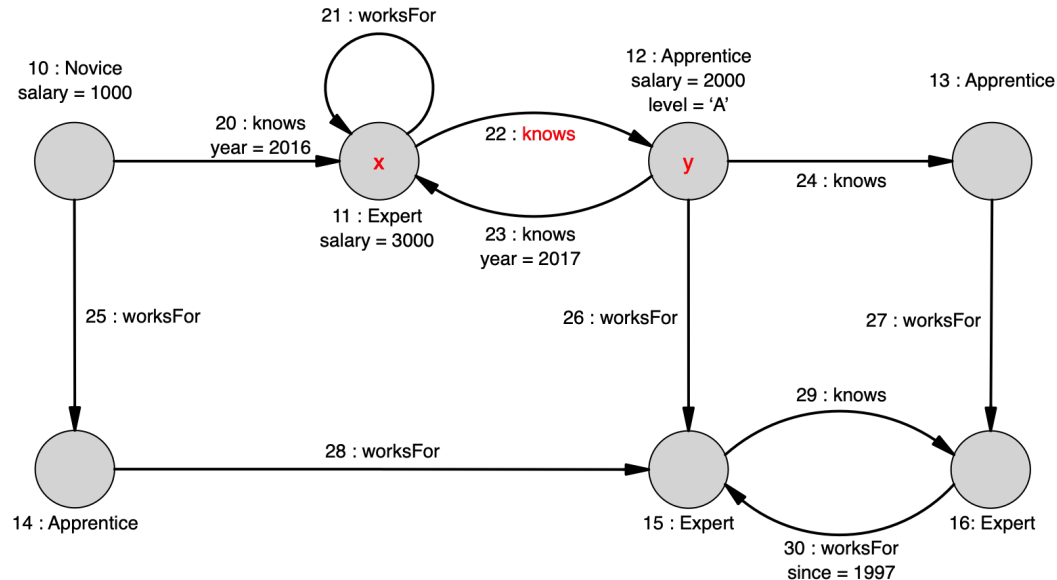
<i>:knownExpert(y, y)</i> IN <i>a</i>	←	<i>:knows(x, y)</i> AS <i>k, :Expert(z), k.year > 2000, y = z.</i>
<i>:related(x, y)</i> IN <i>b</i>	←	<i>:knows(x, y).</i>
<i>:related(x, y)</i> IN <i>b</i>	←	<i>:worksFor(x, y).</i>
<i>result(x)</i>	←	<i>:knownExpert(x, x), :related*(x, y), x.salary < 5000, y.salary < x.salary.</i>



- $\llbracket :knownExpert \rrbracket_G = \{ \{ (11, 11), a \} \}$

Example for RPGLog Evaluation

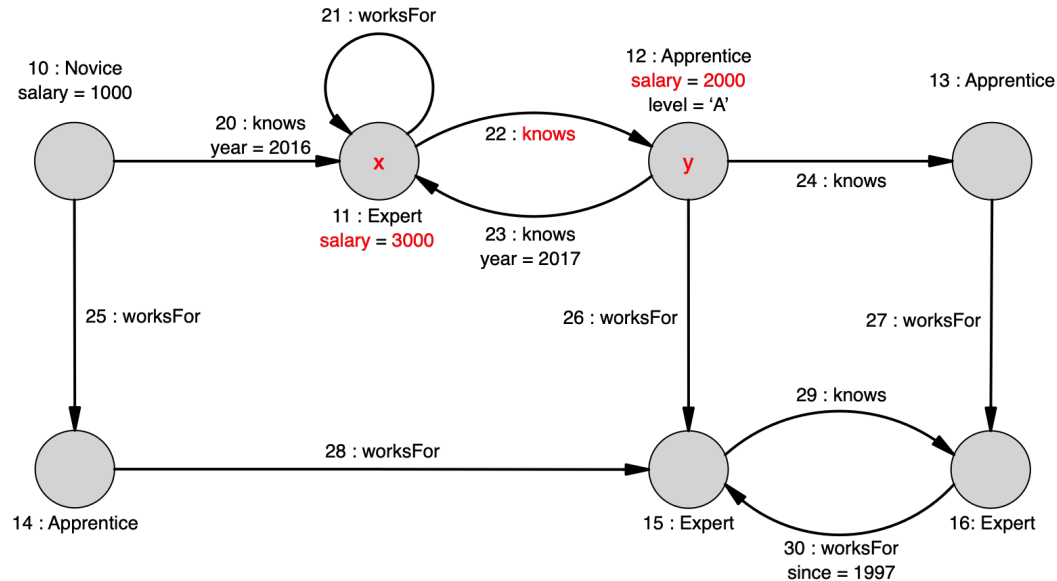
$:knownExpert(y, y)$ IN a	$\leftarrow$	$:knows(x, y)$ AS $k, :Expert(z), k.year > 2000, y = z.$
$:related(x, y)$ IN b	$\leftarrow$	$:knows(x, y).$
$:related(x, y)$ IN b	$\leftarrow$	$:worksFor(x, y).$
$result(x)$	$\leftarrow$	$:knownExpert(x, x), :related^*(x, y),$ $x.salary < 5000, y.salary < x.salary.$



- $\llbracket :knownExpert \rrbracket_G = \{ \{ (11, 11), a \} \}$
- $\{ (11, 12), b \} \in \llbracket :related \rrbracket_G$

Example for RPGLog Evaluation

$:knownExpert(y, y) \text{ IN } a \leftarrow :knows(x, y) \text{ AS } k, :Expert(z), k.year > 2000, y = z.$
 $:related(x, y) \text{ IN } b \leftarrow :knows(x, y).$
 $:related(x, y) \text{ IN } b \leftarrow :worksFor(x, y).$
 $result(x) \leftarrow :knownExpert(x, x), :related^*(x, y),$
 $x.salary < 5000, y.salary < x.salary.$



- $\llbracket :knownExpert \rrbracket_G = \{ \{ (11, 11), a \} \}$
- $\{ (11, 12), b \} \in \llbracket :related \rrbracket_G$
- $\llbracket result \rrbracket_G = \{ (11) \}$

Next aim: An algebraic view of the queries definable in *RPGL*ogic

Next aim: An algebraic view of the queries definable in *RPGL*ogic

Motivation

- ① Use of algebra as a tool in the formal study of path query languages
- ② Use of algebra in the study of engineering of graph queries
 - compilation
 - execution
- ③ Use of a procedural perspective for graph query languages that complements the declarative perspective of the logic

Regular Property Graph Algebra

Next aim: An algebraic view of the queries definable in *RPGL*ogic

Motivation

- ① Use of algebra as a tool in the formal study of path query languages
- ② Use of algebra in the study of engineering of graph queries
 - compilation
 - execution
- ③ Use of a procedural perspective for graph query languages that complements the declarative perspective of the logic

The regular property graph algebra (RPGA) consists of exactly those expressions constructed over $\mathcal{L}$ using transitive closure, union, and graph join operations

Subquery Expressions - Syntax

Grammar for *subquery expressions* *subRPGA*

$$e := \ell \mid e^* \mid e \cup e \mid \bowtie_{\text{pos}_i, \text{pos}_j}^{\Phi, c} (e, \dots, e)$$

Subquery Expressions - Syntax

Grammar for *subquery expressions* *subRPGA*

$$e := \ell \mid e^* \mid e \cup e \mid \bowtie_{pos_i, pos_j}^{\Phi, c} (e, \dots, e)$$

where

- $\ell \in \mathcal{L}$
- $(e, \dots, e)$ is of length $n > 0$
- $c \in \mathcal{C}$ is a context identifier
- $pos_i, pos_j \in \{src_1, trg_1, \dots, src_n, trg_n\}$

Subquery Expressions - Syntax

Grammar for *subquery expressions* subRPGA

$$e := \ell \mid e^* \mid e \cup e \mid \bigwedge_{\substack{\Phi, c \\ pos_i, pos_j}}^c (e, \dots, e)$$

edges

vertices

where

- Φ is a conjunction of a finite number of terms of the form:
 - $\lambda(pos) = \ell$ for $pos \in \{src_1, trg_1, \dots, src_n, trg_n\}$ and $\ell \in \mathcal{L}$
 - $pos_k.p \theta pos_l.q$ or $pos_k.p \theta val$, for
 $pos_k, pos_l \in \{src_1, trg_1, \dots, src_n, trg_n, edge_1, \dots, edge_n\}$, $p, q \in \mathcal{K}$, $val \in \mathcal{N}$ and
 $\theta \in \{=, \neq, <, >, \leq, \geq\}$
 - $pos_k = pos_l$ for $pos_k, pos_l \in \{src_1, trg_1, \dots, src_n, trg_n\}$

Definition

Expressions $e \in RPGA$ are all and only those of the form

$$\bowtie_{\overline{pos}}^{\Phi} (e_1, \dots, e_n)$$

where

- $n > 0$
- each e_i is a *subRPGA* expression for $1 \leq i \leq n$
- Φ is a conjunction of finite number of terms as above
- $\overline{pos}$ is a list of length zero or more, containing elements of $\{src_1, trg_1, \dots, src_n, trg_n\}$.

Example


$$\bowtie_{src_1}^{\emptyset} \left(\bowtie_{trg_1, trg_1}^{\Phi, x} (:knows, (:knows \cup :worksFor)^*) \right)$$

where

- Φ is:

$$\begin{aligned} & (edge_1.year > 2000) \wedge (\lambda(trg_1) = :Expert) \wedge \\ & (trg_1.salary < 5000) \wedge (trg_1.salary > trg_2.salary) \wedge \\ & (trg_1 = src_2) \end{aligned}$$

- x is an arbitrary context identifier

Let $G = (V, E, \eta, \lambda, \nu)$ be a property graph and $e \in \text{subRPGA}$

Regular Property Graph Algebra - Semantics of Subquery Expressions

Let $G = (V, E, \eta, \lambda, \nu)$ be a property graph and $e \in \text{subRPGA}$

Semantics of evaluating e on G is denoted by $\llbracket e \rrbracket_G$

- (case $e = \ell \in \mathcal{L}$). $\llbracket e \rrbracket_G = \{\{\eta(\text{edge}), \text{edge}\} \mid \text{edge} \in E \text{ and } \lambda(\text{edge}) = \ell\}$
- (case $e = (f \cup g)$). $\llbracket e \rrbracket_G = \llbracket f \rrbracket_G \cup \llbracket g \rrbracket_G$
- (case $e = f^*$). $\llbracket e \rrbracket_G = \{(x, x) \mid x \in V\} \cup \{(x, y) \mid x, y \in V \text{ and } x \rightarrow_{\llbracket f \rrbracket_G} y\}$,
where $x \rightarrow_{\llbracket f \rrbracket_G} y$ denotes that for some $n > 0$ there are $e_1, \dots, e_n \in \llbracket f \rrbracket_G$ and $v_1, \dots, v_{n+1} \in V$ such that, $x = v_1$, $y = v_{n+1}$, and, for $1 \leq i \leq n$, $(v_i, v_{i+1}) \in e_i$

Regular Property Graph Algebra - Semantics of Subquery Expressions

Let $G = (V, E, \eta, \lambda, \nu)$ be a property graph and $e \in \text{subRPGA}$

Semantics of evaluating e on G is denoted by $\llbracket e \rrbracket_G$

- (case $e = \bowtie_{pos_i, pos_j}^{\Phi, c} (e_1, \dots, e_n)$). For $1 \leq k \leq n$, let $(src_k, trg_k, edge_k)$ be the schema of $\llbracket e_k \rrbracket_G$, where for each $t = \{(x, y), z\} \in \llbracket e_k \rrbracket_G$, we have $t.src_k = x$, $t.trg_k = y$, and $t.edge_k = z$. Then, we have

$$\llbracket e \rrbracket_G = \{ \{ (t.pos_i, t.pos_j), \omega_G(t.pos_i, t.pos_j, c) \} \mid t \in \sigma_\Phi(\llbracket e_1 \rrbracket_G \times \dots \times \llbracket e_n \rrbracket_G) \},$$

where in evaluating the filter Φ

- $\lambda(pos) = \ell$ is true if and only if $\lambda(t.pos) = \ell$
- $pos_k.p \theta pos_l.q$ is true if and only if $\nu(t.pos_k, p) \theta \nu(t.pos_l, q)$
- $pos_k.p \theta val$ is true if and only if $\nu(t.pos_k, p) \theta val$
- $pos_k = pos_l$ is true if and only if $t.pos_k = t.pos_l$.

Note that σ denotes the standard relational algebra selection operator.

Let $G = (V, E, \eta, \lambda, \nu)$ be a property graph and $e \in \text{RPGA}$

Let $G = (V, E, \eta, \lambda, \nu)$ be a property graph and $e \in RPGA$

Semantics of evaluating e on G is denoted by $\llbracket e \rrbracket_G$

For $e = \bowtie_{pos_1, \dots, pos_m}^{\Phi} (e_1, \dots, e_n)$, we have

$$\llbracket e \rrbracket_G = \{(t.pos_1, \dots, t.pos_m) \mid t \in \sigma_{\Phi}(\llbracket e_1 \rrbracket_G \times \dots \times \llbracket e_n \rrbracket_G)\}$$

Example for Query Optimization

- Consider predicates Ψ built over terms only of the form $\lambda(pos) = \ell$ or $pos.p \theta val$
- Introduce a *value* selection operator $\sigma_{\Psi}(e) := \bowtie_{src_1, trg_1}^{\Psi}(e)$ to *subRPGA*
- Facilitates query optimization heuristic “pushing down selections”

Example for Query Optimization

- Consider predicates Ψ built over terms only of the form $\lambda(pos) = \ell$ or $pos.p \theta val$
- Introduce a *value* selection operator $\sigma_{\Psi}(e) := \bowtie_{src_1, trg_1}^{\Psi}(e)$ to *subRPGA*
- Facilitates query optimization heuristic “pushing down selections”

Example

$$\bowtie_{src_1}^{\emptyset} \left(\bowtie_{trg_1, trg_1}^{\Phi, x} (:knows, (:knows \cup :worksFor)^*) \right)$$

where

- Φ is:

$$\begin{aligned} & (edge_1.year > 2000) \wedge (\lambda(trg_1) = :Expert) \wedge \\ & (trg_1.salary < 5000) \wedge (trg_1.salary > trg_2.salary) \wedge \\ & (trg_1 = src_2) \end{aligned}$$

- x is an arbitrary context identifier

Example for Query Optimization

- Consider predicates Ψ built over terms only of the form $\lambda(pos) = \ell$ or $pos.p \theta val$
- Introduce a *value* selection operator $\sigma_{\Psi}(e) := \bowtie_{src_1, trg_1}^{\Psi} (e)$ to *subRPGA*
- Facilitates query optimization heuristic “pushing down selections”

Example

$$\bowtie_{src_1}^{\emptyset} \left(\bowtie_{trg_1, trg_1}^{\Psi_a, x} \left(\sigma_{\Psi_b} (:knows), (:knows \cup :worksFor)^* \right) \right)$$

where

- Ψ_a is:

$$(trg_1.salary > trg_2.salary) \wedge (trg_1 = src_2)$$

- Ψ_b is:

$$(edge_1.year > 2000) \wedge (\lambda(trg_1) = :Expert) \wedge (trg_1.salary < 5000)$$

- x is an arbitrary context identifier

Equivalence between Algebra and Calculus

RPGLog and *RPGA* express the same query language

Equivalence between Algebra and Calculus

RPGLog and *RPGA* express the same query language

Theorem

For all $q \in \text{RPGLog}$ there is some $e \in \text{RPGA}$ such that for every property graph G it holds that $\llbracket q \rrbracket_G = \llbracket e \rrbracket_G$. For every $e \in \text{RPGA}$ there is some $q \in \text{RPGLog}$ such that for every property graph G it holds that $\llbracket e \rrbracket_G = \llbracket q \rrbracket_G$.

Equivalence between Algebra and Calculus

RPGLog and RPGA express the same query language

Theorem

For all $q \in \text{RPGLog}$ there is some $e \in \text{RPGA}$ such that for every property graph G it holds that $\llbracket q \rrbracket_G = \llbracket e \rrbracket_G$. For every $e \in \text{RPGA}$ there is some $q \in \text{RPGLog}$ such that for every property graph G it holds that $\llbracket e \rrbracket_G = \llbracket q \rrbracket_G$.

Theorem

The evaluation of regular property graph queries is NP-complete in combined complexity and NLOGSPACE-complete in data complexity.

Example for Equivalence

$:knownExpert(y, y) \text{ IN } a \leftarrow :knows(x, y) \text{ AS } k, :Expert(z), k.year > 2000, y = z.$
 $:related(x, y) \text{ IN } b \leftarrow :knows(x, y).$
 $:related(x, y) \text{ IN } b \leftarrow :worksFor(x, y).$
 $result(x) \leftarrow :knownExpert(x, x), :related^*(x, y),$
 $x.salary < 5000, y.salary < x.salary.$

$$\bowtie_{src_1}^{\emptyset} \left(\bowtie_{trg_1, trg_1}^{\Phi, x} (:knows, (:knows \cup :worksFor)^*) \right)$$

where

- Φ is:

$$\begin{aligned} & (edge_1.year > 2000) \wedge (\lambda(trg_1) = :Expert) \wedge \\ & (trg_1.salary < 5000) \wedge (trg_1.salary > trg_2.salary) \wedge \\ & (trg_1 = src_2) \end{aligned}$$

- x is an arbitrary context identifier

$$UCRPQ \subset RPGQ$$

Some Fragments

$$UCRPQ \subset RPGQ$$

$$\textit{positive RA} \subset RPGQ$$

Some Fragments

$$UCRPQ \subset RPGQ$$

$$\text{positive } RA \subset RPGQ$$

Note

positive RA is the fragment of RA obtained by replacing $-$ by $\cap$

Relationship to Graph Query Languages in Tools: G-Core

RPGQ

```
:knownExpert(y, y) IN a ← :knows(x, y) AS k, :Expert(z), k.year > 2000, y = z.  
  :related(x, y) IN b ← :knows(x, y).  
  :related(x, y) IN b ← :worksFor(x, y).  
    result(x) ← :knownExpert(x, x), :related*(x, y),  
                x.salary < 5000, y.salary < x.salary.
```

G-Core

```
PATH    related = (x) - / < :knows:worksFor > / - > (y)  
SELECT  x  
MATCH   (z) - [k:knows] - > (x:Expert) - / < :related* > / - > (y)  
WHERE   k.year > 2000 AND x.salary < 5000 AND y.salary < x.salary
```

Cypher (Overview)

Cypher Facts

- Cypher is a declarative query language for querying graph databases (Property Graph Model)
- Cypher has become the most popular graph query language
- Used by Neo4j and gaining support (as openCypher) in other GDBMS
- GQL (ISO standard) will resemble Cypher

Cypher (Overview)

Cypher Facts

- Cypher is a declarative query language for querying graph databases (Property Graph Model)
- Cypher has become the most popular graph query language
- Used by Neo4j and gaining support (as openCypher) in other GDBMS
- GQL (ISO standard) will resemble Cypher

Example of Cypher Query

```
MATCH (m:Movie)←[:ACTED_IN]-(a:Actor)
WHERE m.title = "Matrix, The"
RETURN COUNT(a)
```

Handwritten red annotations: A checkmark above 'Movie', the word 'edge' above '[:ACTED_IN]', and a checkmark above 'Actor'.

Cypher (Structure)

Cypher queries have the following structure:

Structure of cypher queries

```
[USE]  
[MATCH [WHERE]]  
[OPTIONAL MATCH [WHERE]]  
[WITH [ORDER BY] [SKIP] [LIMIT] [WHERE]]  
RETURN [ORDER BY] [SKIP] [LIMIT]
```

Cypher (Structure)

Cypher queries have the following structure:

Structure of cypher queries

```
[USE]  
[MATCH [WHERE]]  
[OPTIONAL MATCH [WHERE]]  
[WITH [ORDER BY] [SKIP] [LIMIT] [WHERE]]  
RETURN [ORDER BY] [SKIP] [LIMIT]
```

Notice there is no **GROUP BY** in Cypher.

Cypher (Structure)

Cypher queries have the following structure:

Structure of cypher queries

```
[USE]  
[MATCH [WHERE]]  
[OPTIONAL MATCH [WHERE]]  
[WITH [ORDER BY] [SKIP] [LIMIT] [WHERE]]  
RETURN [ORDER BY] [SKIP] [LIMIT]
```

Notice there is no GROUP BY in Cypher.

Cypher Cheat Sheet

For detailed information please see the Cypher Cheat Sheet under <https://neo4j.com/docs/cypher-cheat-sheet/5/all/>

Cypher (Example)

English query

What are the titles of movies that Tom Hanks acted in?

Cypher (Example)

English query

What are the titles of movies that Tom Hanks acted in?

Cypher query

from

```
MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)
WHERE a.name = "Tom Hanks"
RETURN m.title
```

Cypher (Example)

English query

What are the titles of movies that Tom Hanks acted in?

Cypher query

```
MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)
WHERE a.name = "Tom Hanks"
RETURN m.title
```

Cypher (Example)

English query

What are the titles of movies that Tom Hanks acted in?

Cypher query

```
MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)
WHERE a.name = "Tom Hanks"
RETURN m.title
```

Cypher (Example)

English query

What are the titles of movies *that (the actor) Tom Hanks* acted in?

Cypher query

```
MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)
WHERE a.name = "Tom Hanks"
RETURN m.title
```

Cypher (Example)

English query

What are the titles of movies that Tom Hanks acted in?

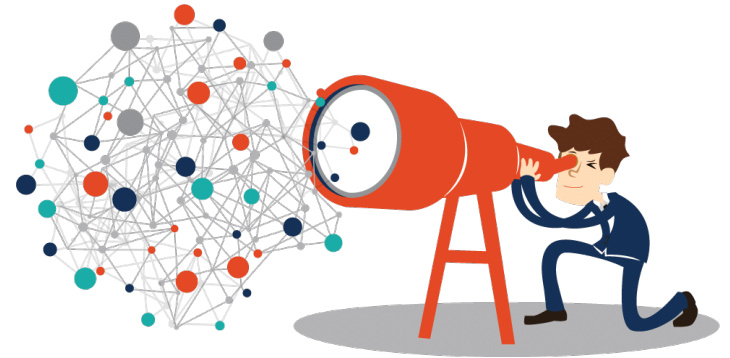
Cypher query

```
MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)
WHERE a.name = "Tom Hanks"
RETURN m.title
```

This is it about Cypher for the moment. Best to use this in practice to learn.

Conclusion

- Rich landscape of query languages for graphs
- Combination of path navigation and pattern matching
- Property Graph Queries are
 - expressive (capable to exploit vertex and edge properties)
 - support declarative and procedural perspectives as logical and algebraic equivalents
 - can be optimized because their containment problems are decidable
- Graph query languages in industry are less expressive



Some References

Graph Query Languages

- Mariano P. Consens and Alberto O. Mendelzon. *GraphLog: A visual formalism for real life recursion*. In PODS, pages 404–416, ACM, 1990. DOI: <http://dx.doi.org/10.1145/298514.298591>
- Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan L. Reutter, and Domagoj Vrgoc. *Foundations of modern query languages for graph databases*. ACM Computing Surveys, 50(5), 2017. DOI: <http://dx.doi.org/10.1145/3104031>
- Renzo Angles, Marcelo Arenas, Pablo Barceló, Peter A. Boncz, George H. L. Fletcher, Claudio Gutierrez, Tobias Lindaaker, Marcus Paradies, Stefan Plantikow, Juan Sequeda, Oskar van Rest, and Hannes Voigt. *G-CORE: A core for future graph query languages*. In SIGMOD, ACM, 2018a. DOI: <http://dx.doi.org/10.1145/3183713.3190654>

Links

- Cypher: <http://www.opencypher.org/>
- Gremlin: <http://tinkerpop.apache.org/>
- Standardization efforts: <http://gqlstandards.org/>